§5.5 Verilog语言程序基本结构

现代电子设计方法EDA技术

Electronic Design Automation

现代电路的设计方法：硬件设计+编程设计

掌握描述方式：硬件描述语言 （verilog）

熟悉设计工具：集成化开发系统

（ Altera（Intel）公司： QuartusII

Xilinx公司： Vivado

Lattice公司： ispDesignEXPERT ）

了解实现的载体：PLD （Programmable Logic Device）

学习方法：重视上机实践

硬件描述语言：描述数字系统的结构、行为、功能和接口

ABEL

AHDL

Verilog HDL

VHDL

} **IEEE标准**

Verilog

IEEE Std 1364-1987
IEEE Std 1364-1995
IEEE Std 1364-2000

Hardware Description Language (HDL)

设计开始之前都应当有一个硬件框图。

verilog 标识符

合法的标识符：

a，fft，and_4，max2uc ，buf$m

非法的标识符：

21A，max#2uc，a-b

- 第一个字符不能是数字；
- 区分大小写，不能和Verilog的保留字相同；
- 关键字必须小写；
- 存盘文件名应与设计的模块名相同。
- 各完整语句均以“;”结尾，注释语句（//，/* */），不参与编译。

verilog 逻辑值

Verilog 语言规定了信号的4种基本逻辑值：

0	逻辑0、逻辑非、低电平
1	逻辑1、逻辑真、高电平
x或X	不确定的逻辑状态
z或Z	高阻态

5.5.1 Verilog模块的基本结构

模块由两部分构成：

- ✓ 描述接口
- ✓ 描述功能

设计模块

```
module      模块名（端口列表）；
input       ;
output      ;      I/O端口模式
wire        ;
reg         ;      信号类型说明
assign      ;（连续赋值）
always @（过程赋值）
            ;（元件例化）
endmodule
```

功能描述

例：一位全加器

模块名

分号

端口列表

端口说明

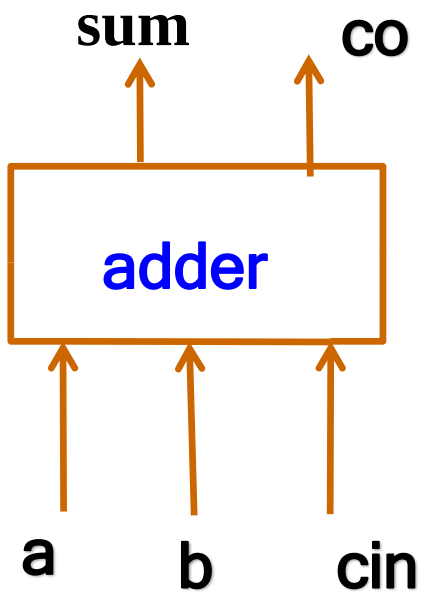
功能描述

无分号

```
module  adder ( a, b, cin , sum,  co ) ;
        input  a , b , cin ;
        output  sum ,  co ;

     assign {co,sum} = a+b+cin;

endmodule
```



注意：

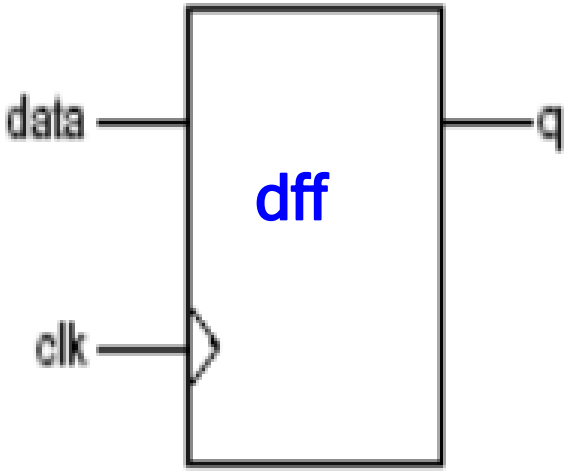
- 模块名必须与文件名相同。
- 模块名不能用中文。
- 数据类型默认wire 型 。

例：一位d寄存器

```
module dff ( clk, data , q ) ;
        input   clk, data;
        output  q;
        reg q ;

      always @ (posedge clk)
        begin
           q <= data ;
        end

endmodule
```



数据类型说明

功能描述

begin end 括起来语句块

5.5.1 Verilog的模块说明及数据类型

模块说明：描述系统的输入、输出端口。

- 模块说明：

```
module  模块名  （端口名）;
```

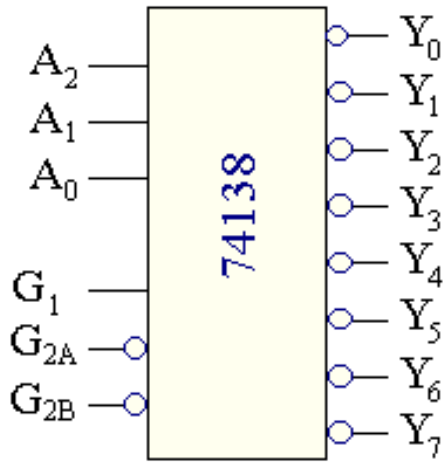
- 端口I/O说明：

```
端口模式  端口名;
```

方向定义(3种)	含义
input	输入 (只读)
output	输出 （在assign中只写）
inout	输入输出 （只能用assign）

```
module  adder ( a, b, cin , sum, co ) ;
        input [2:0]  a,b;     //逻辑矢量
        input   cin;
        output  [2:0]  sum;
        output co;
```

```
module dec38 ( a2,a1,a0,g1,g2a,g2b , y) ;
    input a2,a1,a0,g1,g2a,g2b ;
    output [7:0]  y ;
```



符号

数据类型类型说明:

信号类型　　信号名;

类型定义	含义
wire	线网型
reg	寄存型
parameters	运行时的常数

```
wire  a;
wire [15: 0]  busa;  //16位数据总线
reg  a;
reg [3: 0]  v;
reg [15:0] MEM [0:1023];
      // 1K 个 16 位的存储器
```

- wire: 结构化器件的物理连线。端口如果没有声明，默认为wire型。
- reg：对存储单元或组合逻辑的描述，always 中被赋值的信号必须用reg 类型的变量。
- reg: 输入和双向端口不能声明为reg型。

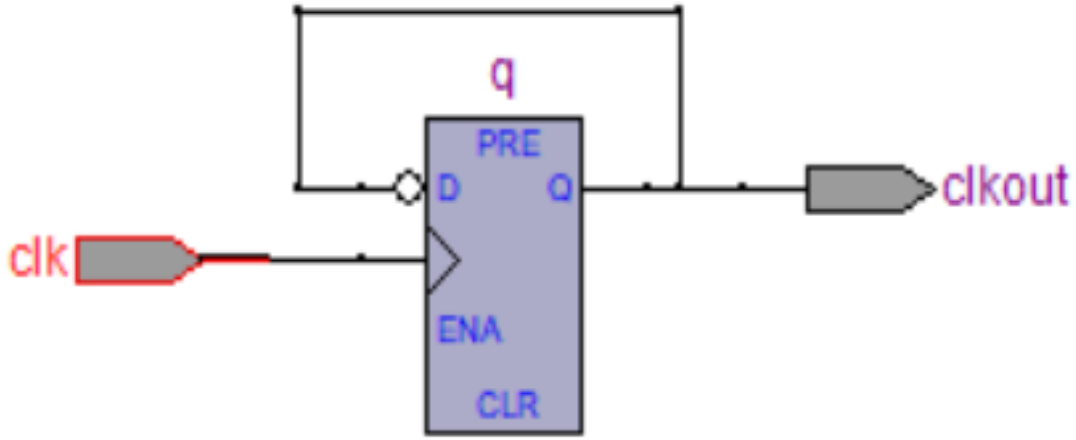
```
module divf (  clk , clkout ) ;   //二分频
        input   clk ;
        output  clkout;
        reg q ;

       always @ (posedge clk)
        begin
            q <= ~q;
        end

       assign  clkout = q ;

endmodule
```

编译之后，点击
Tool>>Netlist Viewers>>RTL Viewers



■ 数据类型类型 parameter 说明:

- parameter：符号常量，其定义只在本模块内有效。
- parameter: 还可以在模块实例引用时，改变引用模块或实例中已定义的参数。

```
module mux (  i1, i2, sel , out ) ;
   parameter width=4  ;
   input sel ;
   input  [width-1:0] i1,i2;       //用参数来说明wire 的位宽
   output [width-1:0]  out ;

   assign   out = sel ? i1 : i2 ;
endmodule
```

- parameter: 可以用参数来表示存储器的大小。

```
parameter wordsize = 16;
parameter memsize = 1024;
reg  [wordsize-1:0]  MEM3 [0: memsize-1];
      // 1024个16位的MEM3存储器
```

数的表示:

<二级制位宽>’ <进制><数字>;

```
16                  // 32位十进制数


8’b0001_0000

8’d16

8’h10


32’bx                       //32位x

2’b?                // ?、z、Z都表示高阻
```

5.5.2 Verilog的 运算符

- 算术操作：+, -, *, /, %（取模）
- 关系操作：>, <, >=, <=, ==（相等）, !=（不等）
- 逻辑操作：&&, ||, !,
- 位操作：~（非）, &, |, ^（异或）, ~^（同或）
- 归约操作：&, ~&, |, ~|, ^, ~^
- 移位操作：<<, >>
- 条件操作：?:
- 连接和复制：{ , }

使用（ ）改变优先级

➢ 逻辑非！、按位反~

!的结果只有一位1，0或x。~的结果与操作数的位数相同。

```
rega = 4'b1011;
bit = ~rega;        // bit = 4’b0100
log = !rega;      // log = 0
```

➢ 逻辑运算、按位运算、归约运算。

逻辑操作、归约操作的结果只有一位。

```
a =  4'b0100;
b  = 4'b0111;;
a && b  ;          //  结果= 1’b1
a & b;       //  结果 = 4’b0100
&a   ;             // 结果= 1’b0，逐位与，判断是否全1
```

➢ 逐位缩减（一元归约）运算符。

单目运算符，对操作数的所有位逐位进行，结果只有一位。

```
& ( ~& )
// ~&与&的运算结果相反
|  ( ~| )
^        // xor
 ~^     //xnor
```

```
rega = 4'b0100;
 & rega ;          //  1'b0，判断是否全1
 | rega ;          //  1'b1，判断是否全0
^ rega ;           //  1'b1，判断1的奇偶
 ~& rega;          // ~(0&1&0&0) = 1'b1，判断是否有0
 ~| rega;          // ~(0|1|0|0) = 1'b0，判断是全0
```

➢ 移位运算符。

>> 右移 、<< 左移。

```
a  = 8'b11010001;
a  << 4 ;          // a = 8'b00010000

b  = 8'b11010001;
b  >>  4 ;         // b = 8'b00001101
```

➢ 位拼接运算符。

{ } ，将两个或多个信号的某些位拼接起来。不允许连接非定长常数。

```
wire [5:0]  a, b ;
wire [1:0] c ;
wire [5:0] d ;
assign a = 6'b101101 ;
assign b = 6'b111000 ;
assign c = 2'b11 ;
assign d ={a[5], b[2:0] , c} ;  // d=100011
```

×

```
assign b = { d ,1 }
//不允许拼接非定长常数
```

。

运算符说明：

- 自动调整位宽。运算表达式结果的长度由最长的操作数决定。
- 操作结果的长度：由赋值左端目标长度决定。

```
wire [3:0]  arc, bar, crt;
wire [5:0]  frx;
assign arc = bar + crt;      //  长度4位，加法的溢出部分被丢弃
assign frx = bar + crt;       //长度6位，任何溢出的位存储在frx [ 4 ]中

4'b0110 ^ 5'b10000   // 结果  5' b10110
```

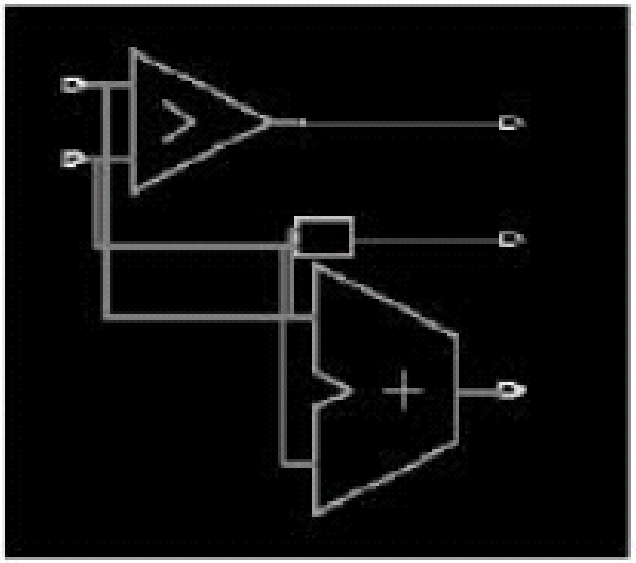
- 位操作、归约操作和逻辑操作

a = 1011
b = 0010

bit-wise	unary reduction	logical
a | b = 1011	| a = 1	a || b = 1
a & b = 0010	& a = 0	a && b = 1

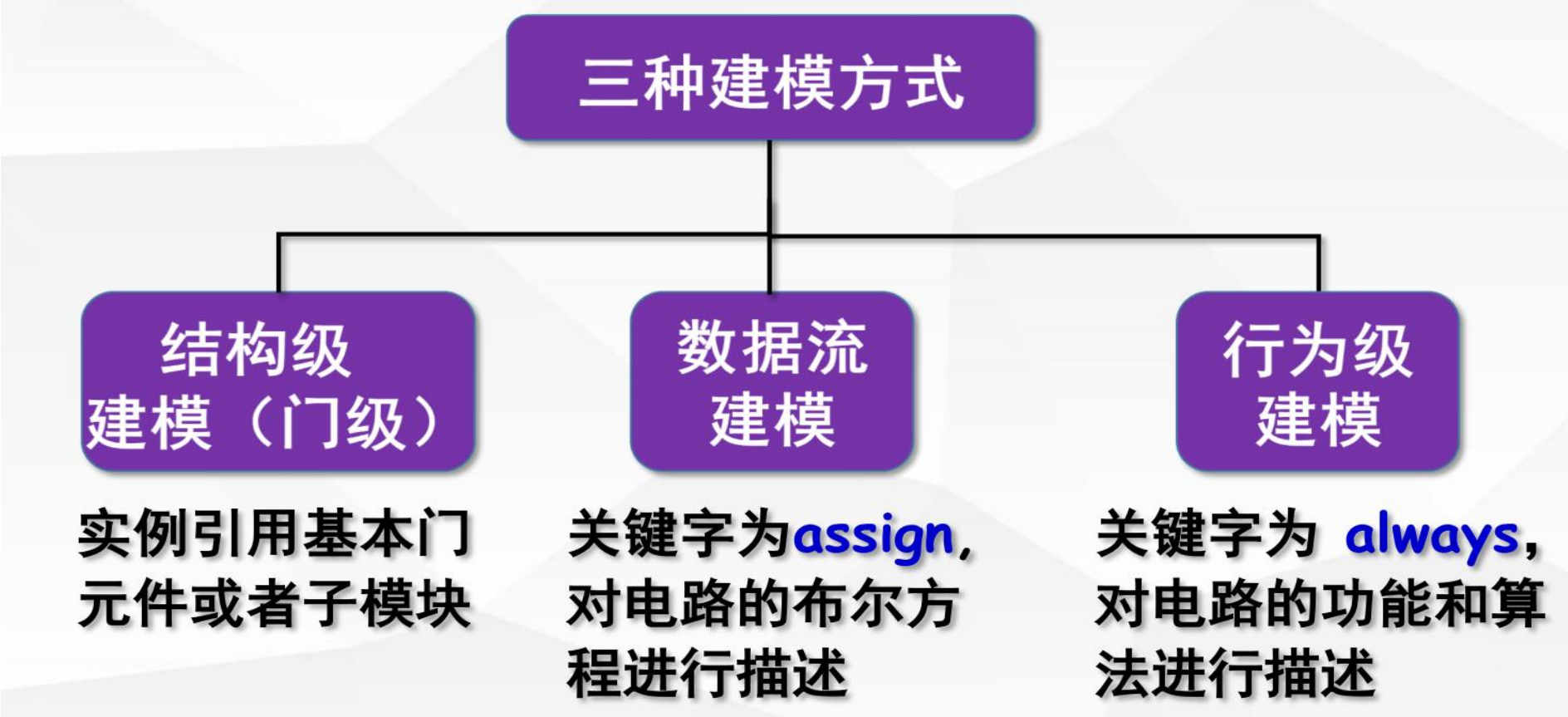
- 算术和关系操作

```
assign  c = a + b ;
assign  bigger = a > b ;
assign  eq = ( a == b ) ;
```



5.5.3 Verilog的功能描述语句

功能描述：assign，always，元件例化，是并发语句。

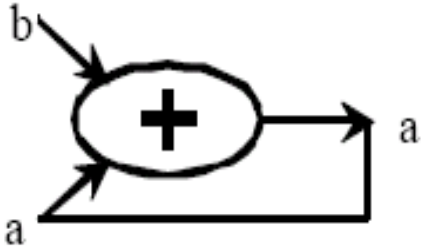


一、 连续赋值 assign 语句：

assign 赋值目标 = 表达式；

- 连续赋值运算符 = 。
- assign后面只能是一个表达式。
- 更新赋值，避免出现反馈。

assign a = b + a ; ×



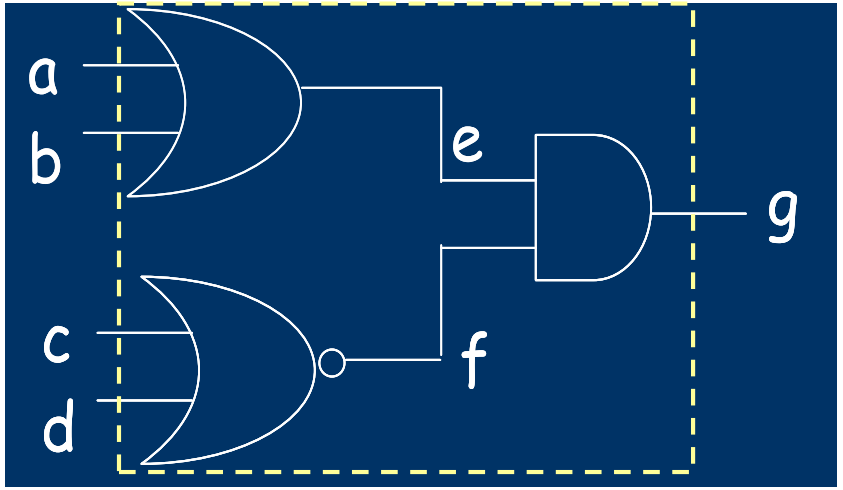
- 在assign语句中输出引脚是只写，inout型引脚只能用在assing语句中。
- 赋值目标必须是wire型的。
- 多个 assign 之间是并行关系。
- assign 只能逐句使用，只能完成简单的组合逻辑。
- 不能对同一个信号进行assign 多次赋值（多源驱动） 。

```
module exp1(  a ,b ,c ,d , g  ) ;

input  a , b ,c ,d ;
output   g ;
wire  e , f ;

assign  g = e & f  ;
assign  f = ~ (c | d)  ;
assign  e = a | b  ;

endmodule
```



并行行为

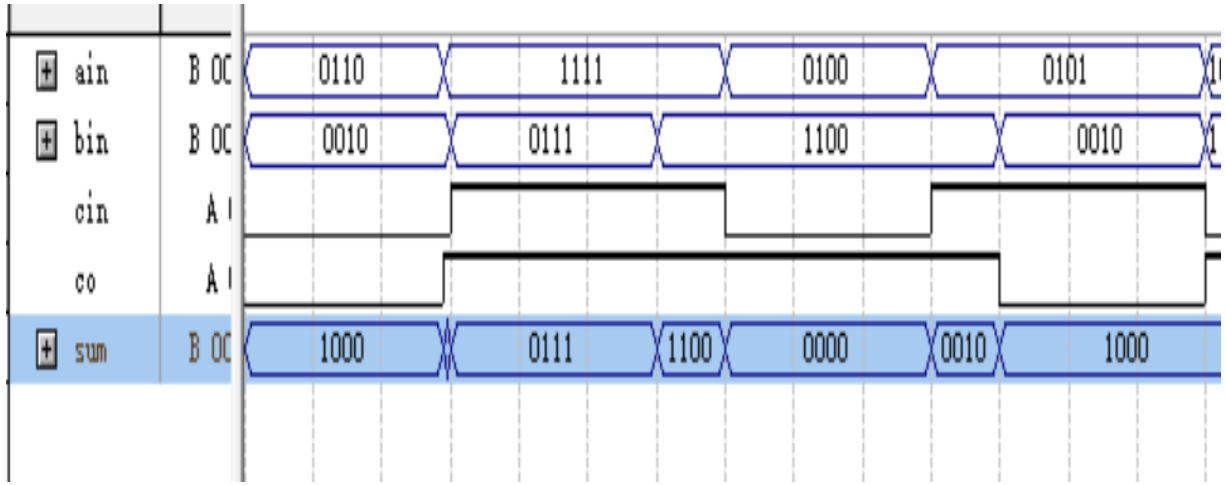
实现 四位二进制全加器

```
module adde ( ain ,bin ,cin , sum, co  );

input [3:0]  ain , bin ;
input  cin ;
output [3:0]  sum ;
output co ;

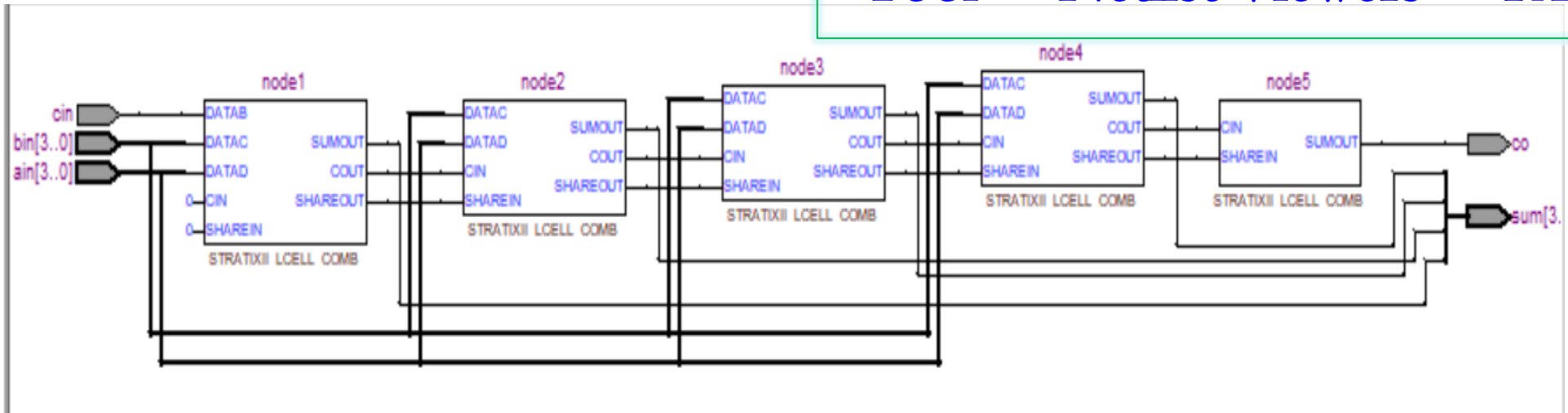
assign {co , sum } =  ain+bin+cin ;

endmodule
```



编译之后，点击

Tool>>Netlist Viewers>>RTL Viewers



二、过程赋值always语句块：

（1）语法：

过程赋值语句 always 后面是语句块。可描述所有的功能。

赋值目标必须是 reg 型的。

```
always @(敏感信号表)
  begin
        =      ;
       <=      ;
     if 语句
     case 语句
  end
```

➢ 过程赋值运算符

- = ，阻塞赋值。立即更新数值。
- <= ，非阻塞赋值。延迟更新数。所有的赋值语句同时完成更新。
- 组合逻辑用 = 赋值。
- 时序逻辑用 <= 赋值 。

敏感信号表：

- ✓ 敏感信号的变化才能启动进程。
- ✓ 组合逻辑中，所有输入都作为敏感信号,否则仿真结果和综合结果会不一致。
- ✓ 边沿敏感、电平敏感。

```
always @ (敏感信号表)
  begin
      =      ;
     <=      ;
    if 语句
    case 语句
  end
```

上升沿触发：

```
always @(posedge 信号名)
```

下降沿触发：

```
always @(negedge 信号名)
```

电平触发：

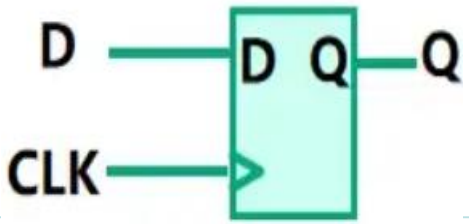
```
always @(信号名)
```

```
always @ ( reset or set or d)
always @ ( reset , set , d)
```

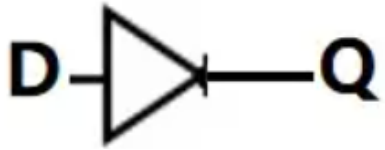
所有输入信号均触发：（组合逻辑）

```
always @( * )
```

```
always @ (posedge CLK)
 begin
     Q = D ;
 end
```




```
always @ (D)
 begin
     Q = D ;
 end
```



实现 一位二进制全加器。

```
module adder ( a , b, cin, s, cout );

input a , b, cin;
output s, cout;
 reg s , cout;

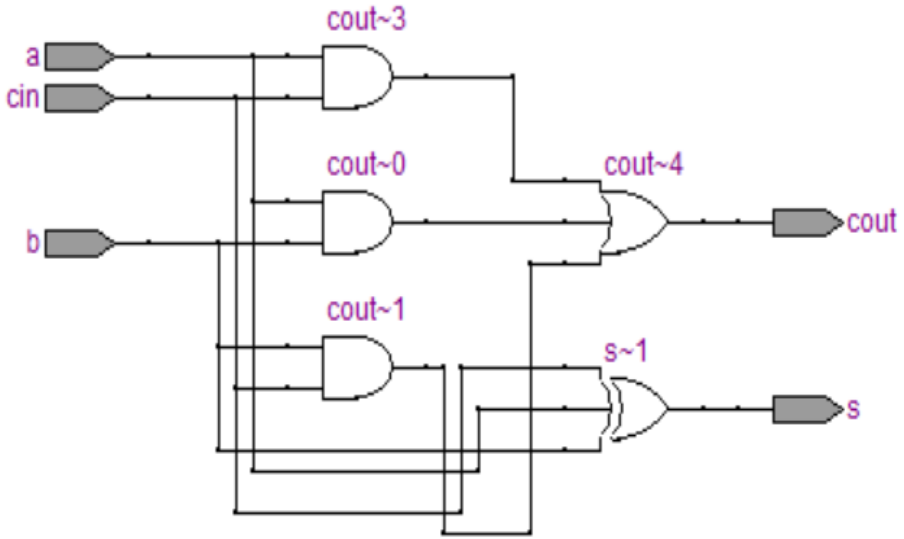
always @ ( *)
   begin
     s = a^b^cin ;
     cout = a&b | b&cin | a&cin :
   end

endmodule
```

reg 数据类型

组合逻辑通配符 *

顺序执行




```
module exp ( a , b, cin, s, cout );

 input a , b, cin;
 output s, cout;

asssign  cout = a&b | b&cin | a&cin ;
assign   s = a^b^cin ;

endmodule
```

wire 数据类型

并发执行

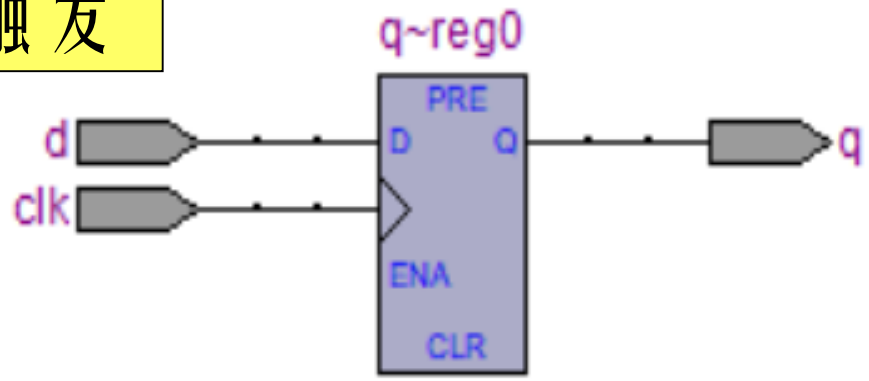
```
module qff ( clk, d, q );

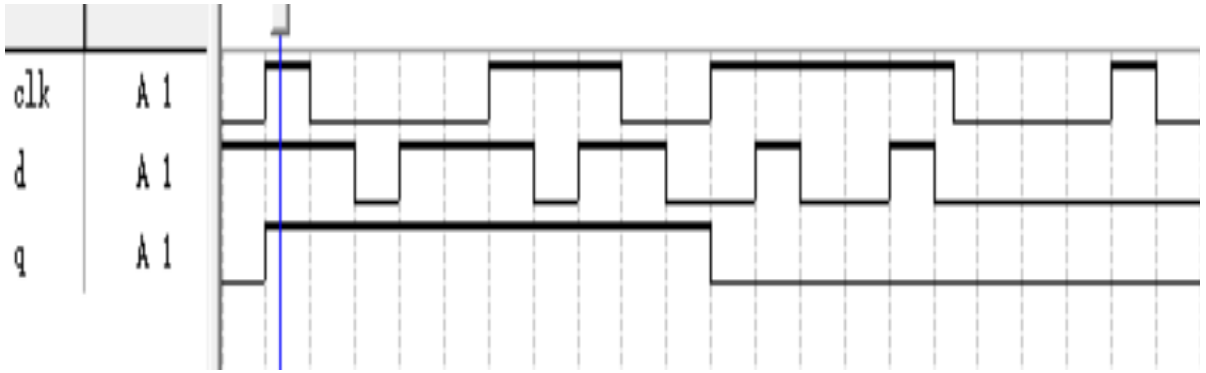
input clk, d;
output q ;
 reg q;

 always @ (  posedge clk )
    begin
          q <= d ;
     end
 endmodule
```

时钟上升沿触发

综合后生成寄存器（register）





（2）阻塞赋值与非阻塞赋值的比较

注意：

- ✓ 不要在组合逻辑中使用 <= 赋值；
- ✓ <=用于实现时序逻辑；
- ✓ 在同一个always块里不要混用两种赋值语句；

■ 阻塞赋值

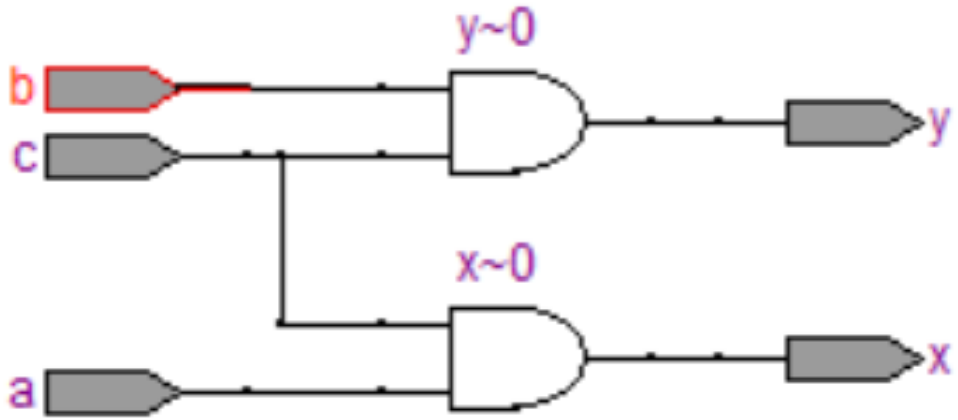
在实现组合逻辑的always块里用阻塞赋值

```
module expe ( a , b, c , x, y );

 input a , b, c;
 output y , x;
 reg d, x, y;

 always @ ( * )
    begin
      d = a;
      x = c & d ;
      d = b;
      y = c & d ;
    end

 endmodule
```



- - 结果: x = ca
y = cb

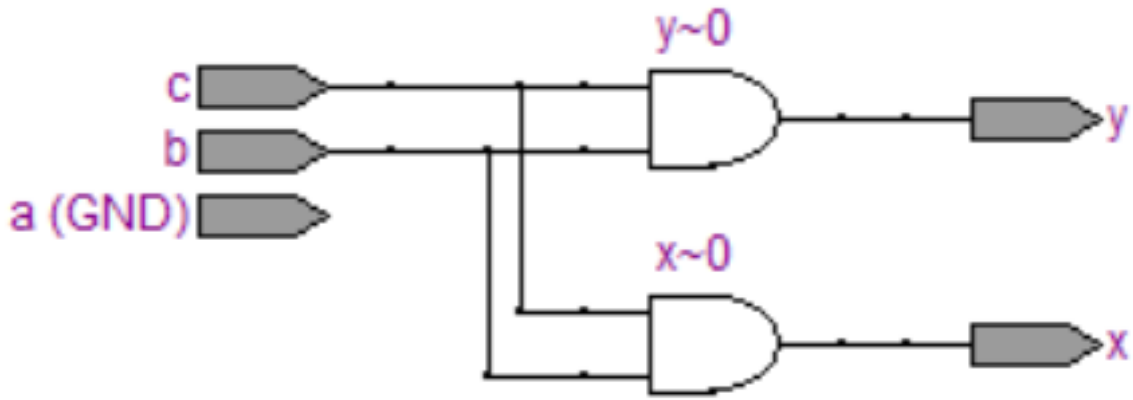
```
module expe ( a , b, c , x, y );

 input a , b, c;
 output y , x;
 reg d, x, y;

always @ ( * )
   begin
     d <= a;
     x <= c & d ;
     d <= b;
     y <= c & d ;
   end

endmodule
```

■非阻塞赋值



读出（begin中）

更新（end后）

不要在always中对同一信号多次非阻塞赋值

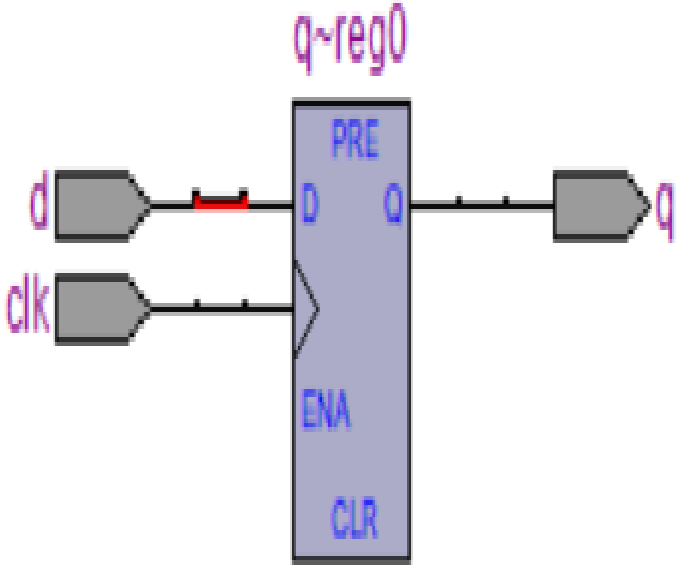
-- 结果: x = cb
y = cb

```
module qff  ( clk, d , q );

 input clk, d;
 output q ;
 reg a, b, q;

 always @ ( posedge clk)
    begin
         a = d ;
         b = a ;
         q = b ;
     end
 endmodule
```

综合后生成一位寄存器



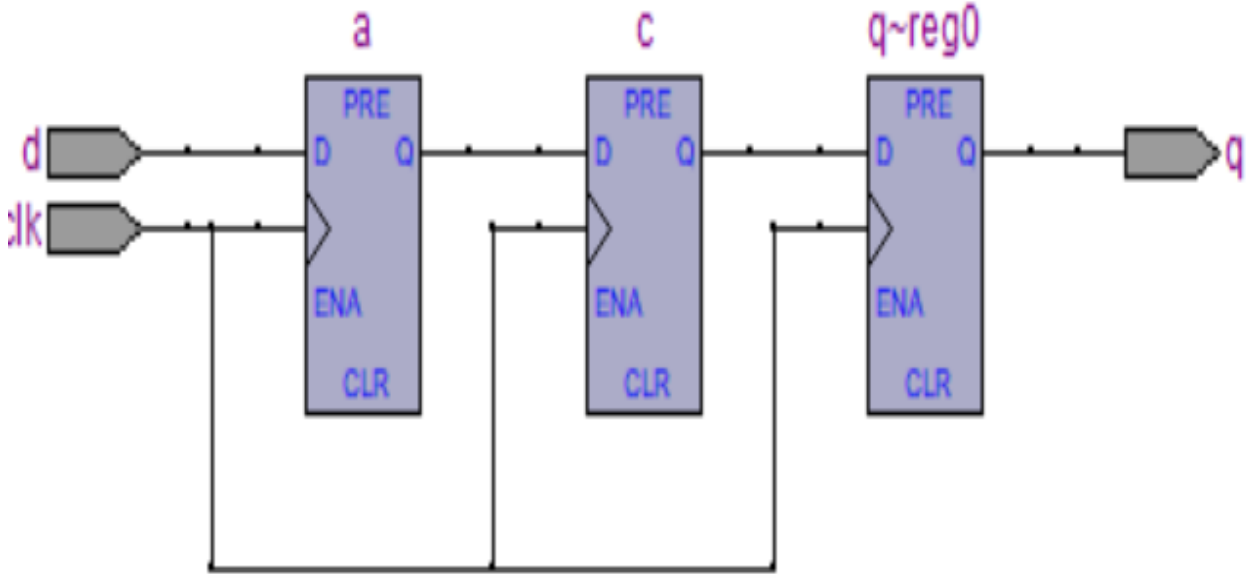
阻塞赋值

```
module qff  ( clk, d , q);

 input clk, d;
 output q ;
 reg a, c, q;

 always @ ( posedge clk)
    begin
        a <= d ;
        c <= a ;
        q <= c ;
     end
 endmodule
```

综合后生成移位寄存器



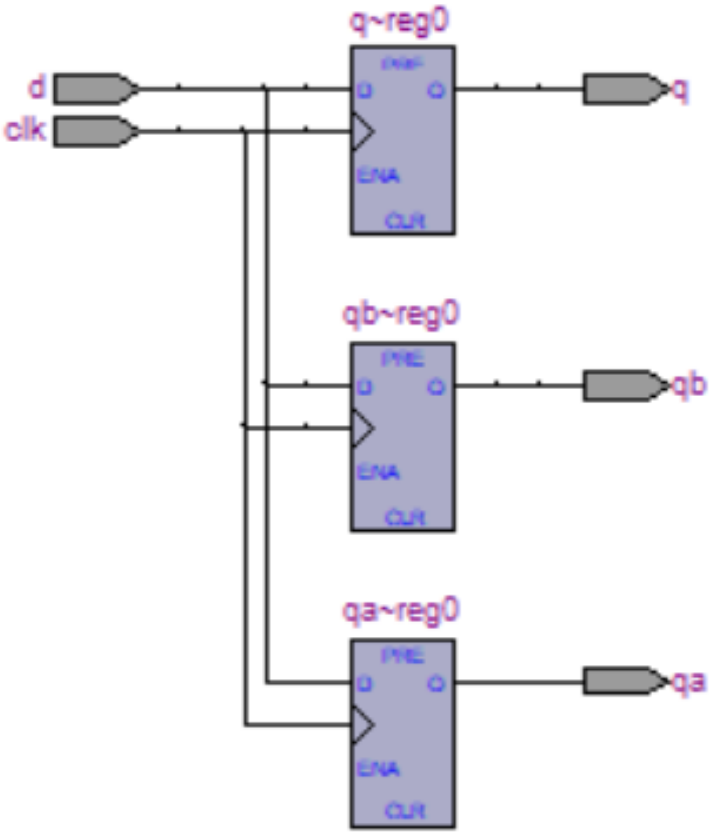
非阻塞赋值

```
module qff  ( clk , d ,q ,  qa , qb );

 input clk, d;
 output qa ,qb , q  ;
 reg qa, qb, q;

 always @ ( posedge clk)
    begin
         qa = d ;
         qb = qa ;
         q = qb ;
     end
endmodule
```

综合后生成3个寄存器并联



阻塞赋值，

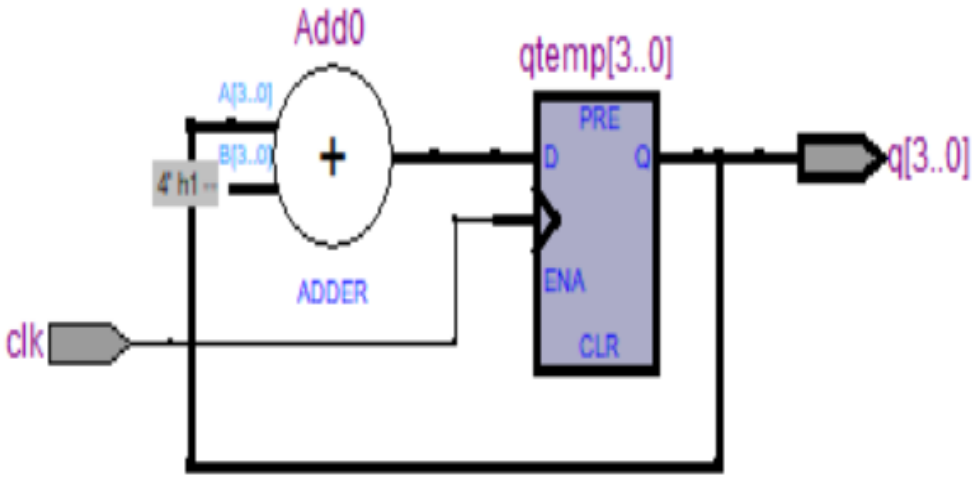
输出引脚，

存储器不要使用阻塞赋值

(3) assign 和 always 并发

注意:

- ✓ 不要在一个aways 中同时使用 = ， <= 赋值；
- ✓ 不要对一个信号多次驱动;
- ✓ 信号可以在多个always 、assign 间传递;
- ✓ 不能在多个always，与assign中对同一个信号多次赋值




```
module  count4 ( clk , q );

    input  clk;
    output [3:0]  q;
    reg [3:0]   qtemp ;

     always @ (posedge clk)
       begin
          qtemp <= qtemp+1;
       end

    assign q = qtemp ;

endmodule
```

4位二进制计数器

(4) 转向控制语句

通过条件控制决定是否执行一条或几条语句，或重新执行一条或几条语句，仿真时顺序进行。

主要有:

if 语句、case 语句、for 语句



if 、case、for 语句必需在 always 块中。

1) if 语句

• if 语句的门闩控制（不完全if ）

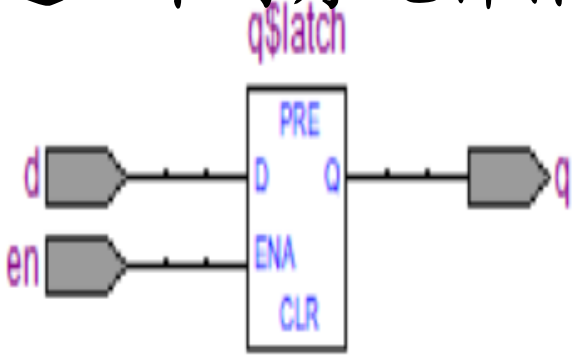
对于不完全的if语句，verilog综合器将引进一个时序元件保持当前状态值。用于锁存器。不建议用不完全的if。

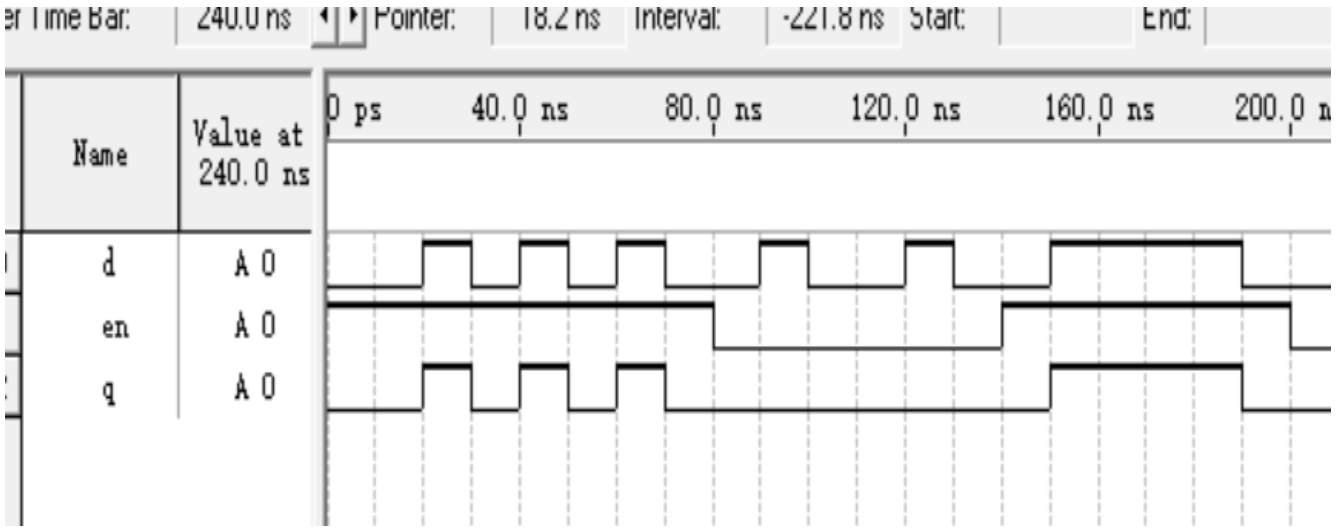
```
module latch ( en,  d , q );

input en, d;
output q ;
reg q;

always @ ( en, d )
   begin
     if ( en )
        q <= d ;    // else  q <= q ;
    end
endmodule
```

电平触发



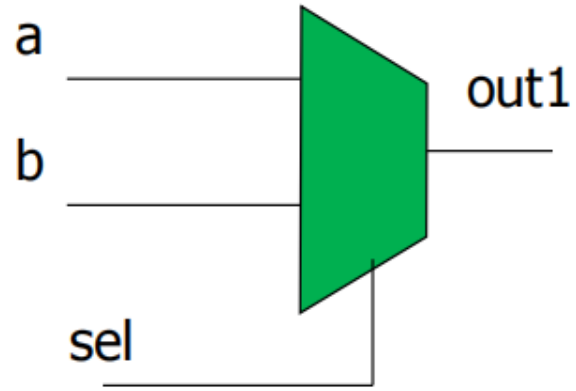


2) if else 语句

- if 语句中最先出现的条件优先级最高（自上而下优先）。
- 可以有多个else if，但必须有一个else（组合逻辑，所有分支条件都要对变量赋值）。
- 良好的代码风格，建议写成完整的 if · · · · · else 结构

```
if （条件表达式1)
    begin
        语句块1;
    end
else if (条件表达式2 )
    begin
        语句块2;
    end
else if  (条件表达式3 )
    begin
        语句块3;
    end
else
    begin
        语句块n;
    end
```

例：2选1选择器。




```
module mux2_1  (  a, b, sel , out1 );

input a , b, sel;
output out1;
reg  out1;

always @ ( * )
  begin
    if ( sel )
      out1 = a ;
    else
      out1 = b ;
  end

endmodule
```

```
module mux2_1  ( a, b, sel , out1 );

  input a , b, sel;
  output out1;

assign  out1 = sel ? a , b ;

endmodule
```

3) case 语句

根据某个表达式的值来选择执行体。 无优先级。

- 选择表达式必须使用（ ）。
- 分支条件须在表达式范围内，且不能重合。
- 执行时必须选中且只能选中一个分支。
- 组合逻辑，每个条件都有赋值，描述真值表。
- 所有值必须列举穷尽，组合、时序都写

default。

```
case (选择表达式)
    值1:
        begin
         语句块1;
        end
    值2:
        begin
         语句块2;
        end
 . . . . . .
   default:
        begin
         语句块n+1;
        end
  endcase
```

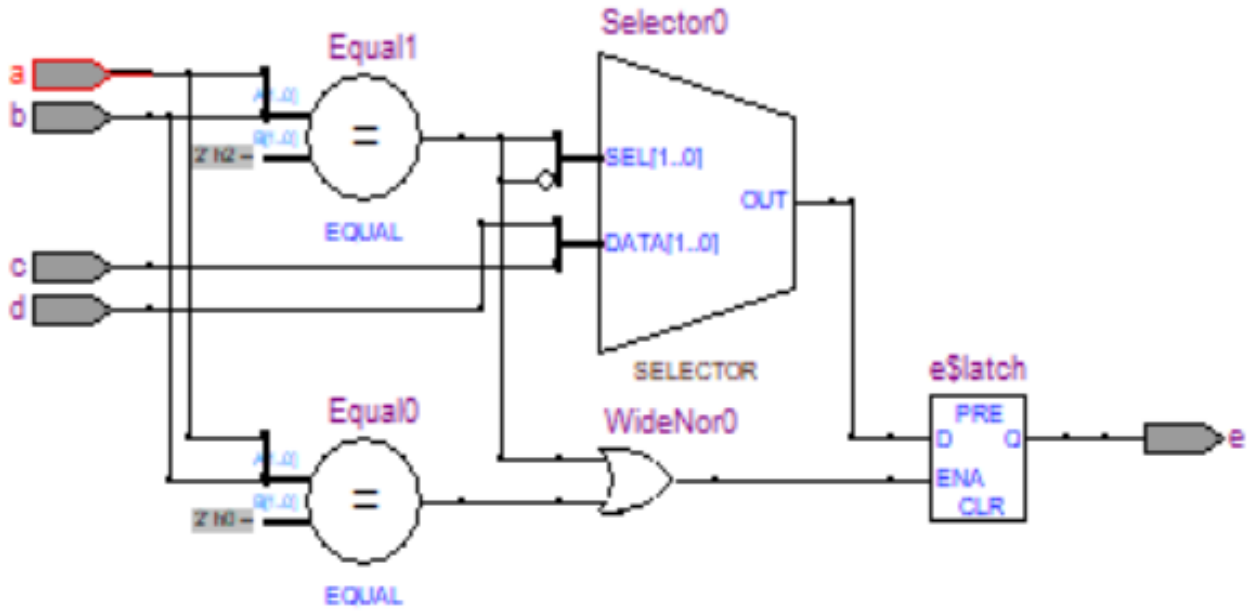
```
module mux2 ( a , b , c , d , e ) ;

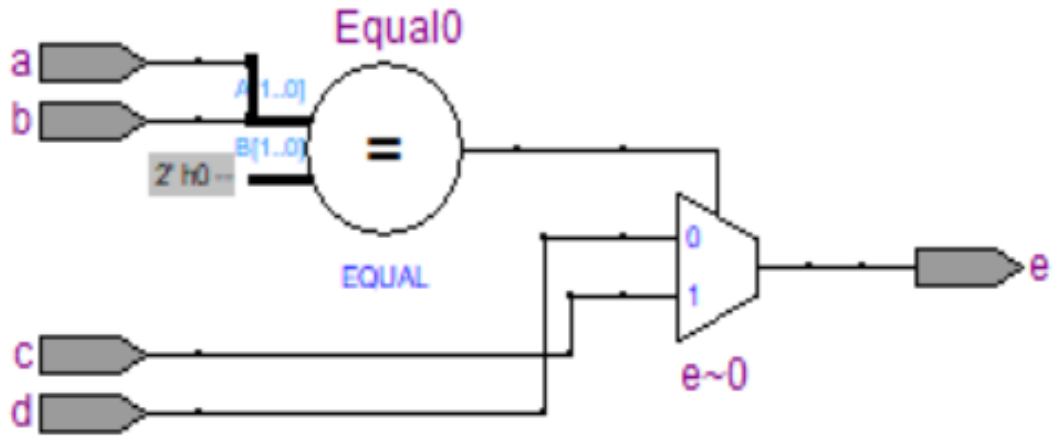
input a , b , c ,d  ;
output e;
reg e ;

always @ ( * )
  begin
     case ( {a,b} )
        2'b00 : e = c ;
        2'b10 : e = d ;
     endcase
  end

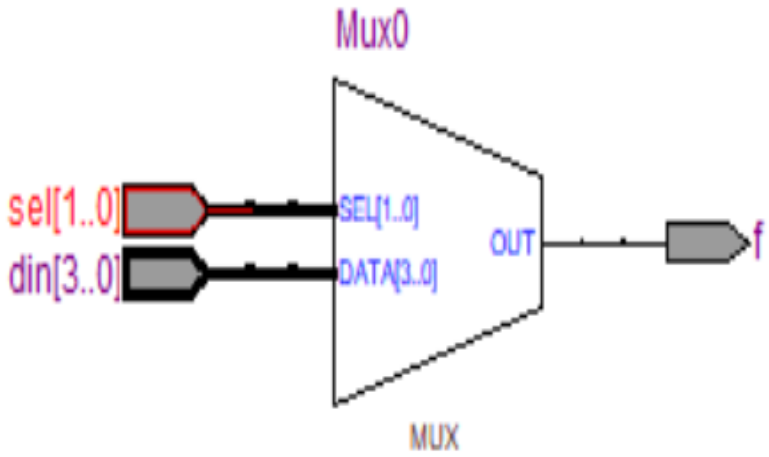
endmodule
```

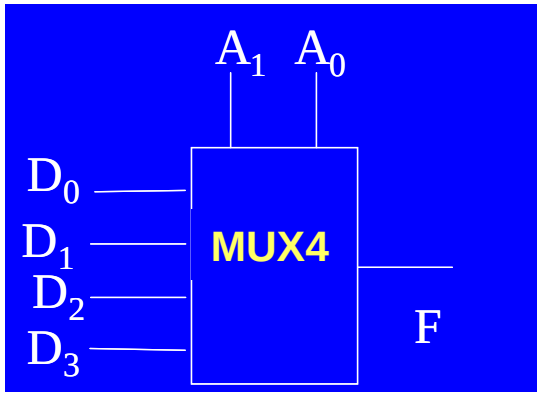
default: e = d ;





例　4选1多路选择器描述方式






```
module mux4 ( sel , din , f) ;

 input [1:0] sel;
 input [3:0] din ;
 output f ;
 reg f;

  always @ ( * )
    begin
       case (sel)
          2'b00:    f = din[0] ;
          2'b01:    f = din[1] ;
          2'b10:    f = din[2] ;
          default:  f = din[3] ;
       endcase
    end

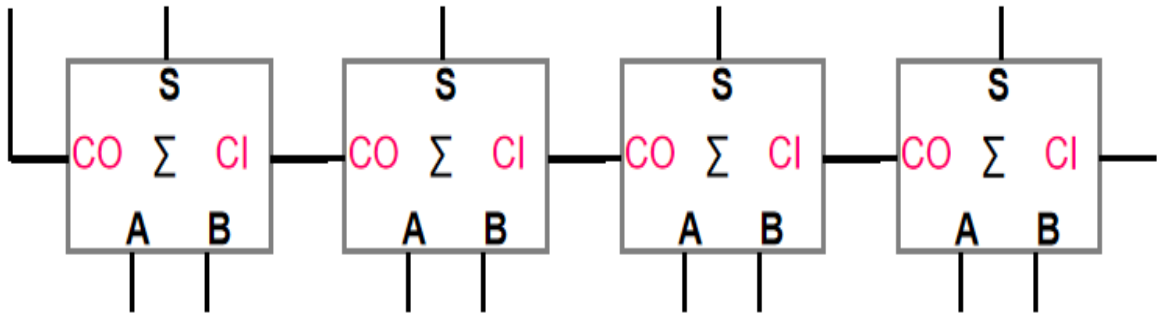
endmodule
```

4) for 语句 (不提倡使用）

```
for (表达式1，表达式2，表达式3)
   begin
        语句块;
            …   ..   ..    …
   end
```

- 表达式1：设置循环变量初值，必须定义一个 i 循环变量;
- 表达式2：循环执行判断条件;
- 表达式3：更新循环变量，只能用类似 i=i+1 或 i=i-1;
- 循环次数不定的循环，无法被综合 ;
- 注意：初值、步长、循环条件中不能有变量 。

例 n位串行加法器




```
module adde (  ain , bin , cin , sum , co  ) ;

parameter n=4 ;
input [n-1:0] ain , bin  ;
input cin ;
output [n-1:0] sum ;
output co ;
reg [n-1:0] sum ;
reg co ;
reg [n:0] ci ;
integer k ;  // 32根线
```

```
always @ ( * )
  begin
     ci[0] = cin ;
     for ( k=0 ; k<n ; k=k+1 )
        begin
           sum[k] = ain[k] ^ bin[k] ^ ci[k] ;
           ci[k+1] = ain[k] & bin[k] | (ain[k]^bin[k])&ci[k] ;
        end
      co = ci[n] ;
  end

endmodule
```

5.5.3 verilog 语言的描述风格

三种描述风格：

行为描述：always语句，使用功能描述

数据流描述：assign语句 ,使用布尔代数式描述

结构化描述：元件例化

5.5.4 基本逻辑电路的VHDL设计

一、组合电路

原则1：在always中用到的所有输入信号都出现在敏感信号列表中；

原则2：电路的真值表必须在代码中完整的反映出来（使用default），否则会生成锁存器；

原则3：使用if•• else••，必须完全互斥；

原则4：不能在一个表达式中对某个信号即读又写（不能反馈）；

原则5：不能使用 <= 非阻塞赋值。

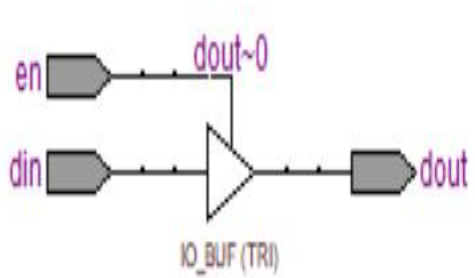
1. 三态门及总线缓冲器

```
module tri  (din , en , dout ) ;

 input din , en ;
output dout ;

 assign dout = en ? din: 1'bz;

endmodule
```

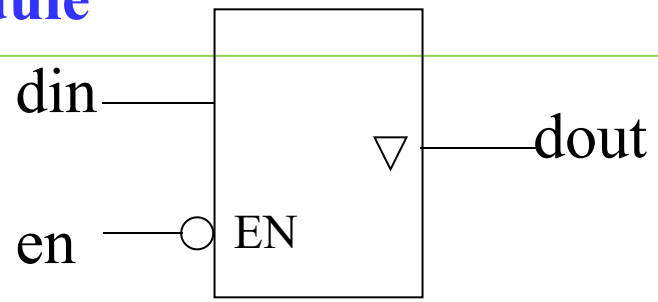


8位数据总线?

```
module tri  (din , en , dout ) ;

 input din , en ;
 output dout ;
 reg dout ;

 always @ (en , din)
   begin
     if ( !en)
        dout = din ;
     else
        dout = 1'bz ;
   end
endmodule
```



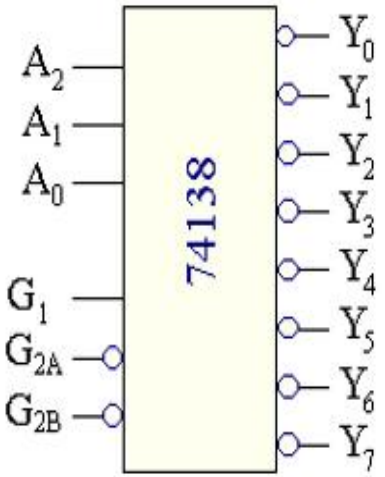
2. 3：8译码器

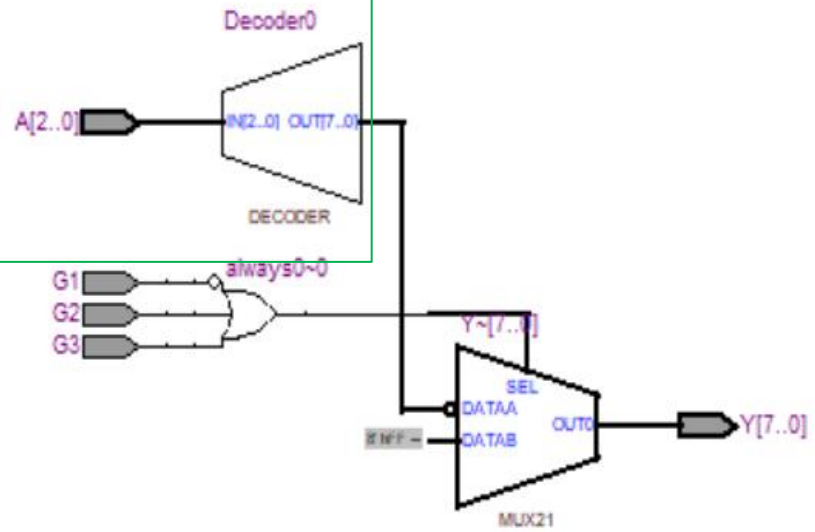
```
module decode3_8( A , G1,  G2 ,G3 , Y  );
input G1, G2, G3;
input [2:0] A ;
output [7:0]  Y ;
reg [7:0] Y ;
reg s;

always @ ( * )
   begin
      s = G2 | G3 ;
       if (G1 == 0 || s== 1)
           Y = 8'b1111_1111;
        else
          begin
```

使能端 if

```
            case ( A )
                  3'b000: Y = 8'b1111_1110;
                   3'b001: Y = 8'b1111_1101;
                   3'b010: Y = 8'b1111_1011;
                   3'b011: Y = 8'b1111_0111;
                   3'b100: Y = 8'b1110_1111;
                   3'b101: Y = 8'b1101_1111;
                   3'b110: Y = 8'b1011_1111;
                   3'b111: Y = 8'b0111_1111;
                   default:Y = 8'b1111_1111;
            endcase
          end
    end
endmodule
```





3. 8-3优先编码器

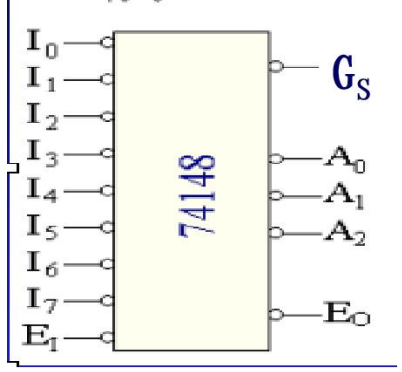
```
module encode_83( I ,EI ,A, GS ,EO );

input [7:0] I ;
input EI ;
output [2:0] A ;
output  GS , EO ;
reg [2:0]  A ;
reg GS, EO ;

always @ ( * )
  begin
    if  (EI )
      begin
        A = 3'b111 ;
        GS = 1’b1 ;
        EO = 1’b1 ;
      end
```

```
    else if (I[7] == 0)
      begin
        A = 3'b000;
        GS = 1’b0;
        EO = 1’b1 ;
      end
……….……….……….……….……….……
    else if (I[0] == 0)
      begin
        A = 3'b111;
        GS =1’b 0;
        EO = 1’b1;
      end
    else
      begin
        A = 3'b111;
        GS = 1’b1;
        EO = 1’b0;
      end
  end

endmodule
```



输入								输出			
$\overline{E_I}$	$\overline{I_7}$	$\overline{I_6}$	$\overline{I_5}$	$\overline{I_4}$	$\overline{I_3}$	$\overline{I_2}$	$\overline{I_1}$	$\overline{I_0}$	$\overline{Y_2}$	$\overline{Y_1}$	$\overline{Y_0}$	$\overline{G_S}$	$\overline{E_O}$
1	×	×	×	×	×	×	×	×	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	×	×	×	×	×	×	×	0	0	0	0	1
0	1	0	×	×	×	×	×	×	0	0	1	0	1
0	1	1	0	×	×	×	×	×	0	1	0	0	1
0	1	1	1	0	×	×	×	×	0	1	1	0	1
0	1	1	1	1	0	×	×	×	1	0	0	0	1
0	1	1	1	1	1	0	×	×	1	0	1	0	1
0	1	1	1	1	1	1	0	×	1	1	0	0	1
0	1	1	1	1	1	1	1	0	1	1	1	0	1

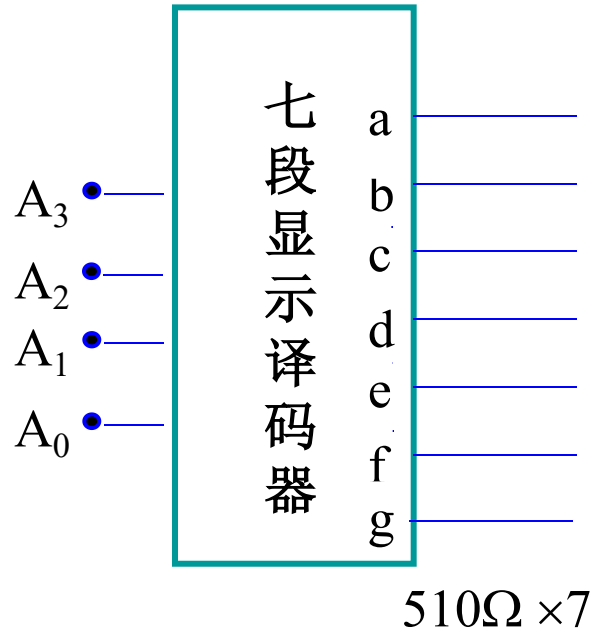
4. 七段显示译码器

```
module seg_7(  A, seg );

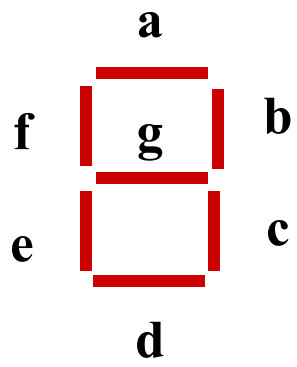
input [3:0] A ;
output [6:0]  seg ;
reg [6:0]  seg ;

always @ (  *  )
   begin
     case (A)
        4'd0:  seg = 7'b111_1110 ;
        4'd1:  seg = 7'b011_0000 ;
        4'd2:  seg = 7'b110_1101 ;
        4'd3:  seg = 7'b011_0000 ;
        4'd4:  seg = 7'b011_0011 ;
        4'd5:  seg = 7'b101_1011 ;
        4'd6:  seg = 7'b101_1111 ;
        4'd7:  seg = 7'b111_0000 ;
        4'd8:  seg = 7'b111_1111 ;
        4'd9:  seg = 7'b111_1011 ;
        default:  seg = 7'b000_0001 ;
     endcase
end

endmodule
```







5. 8位奇偶校验位产生电路

```
module odd_che  ( bus , check  ) ;

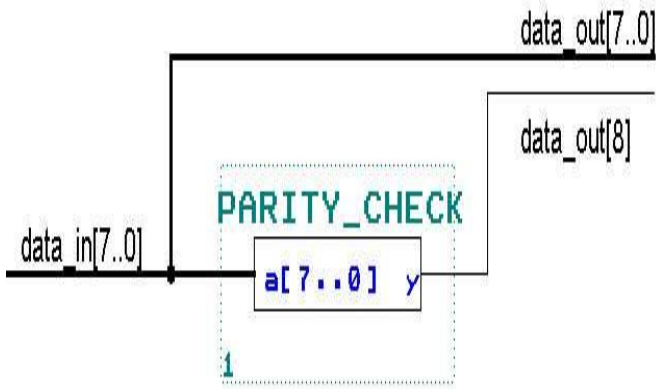
 input [7:0] bus ;
 output check ;

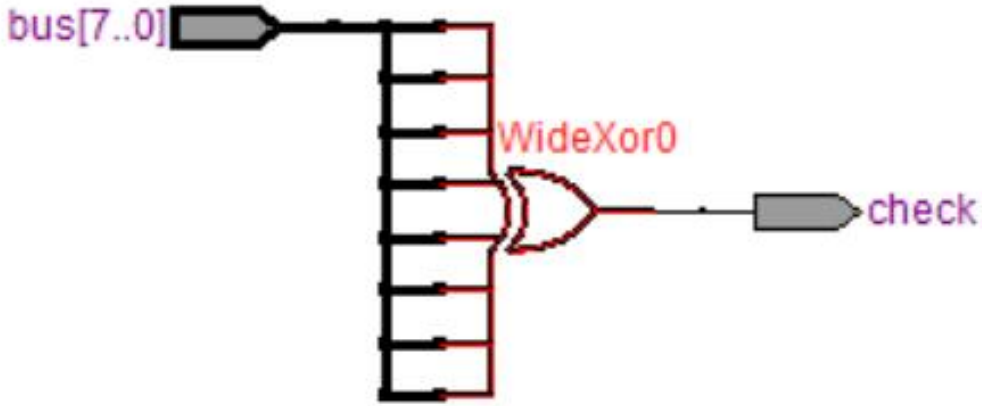
 assign check = ^bus ;

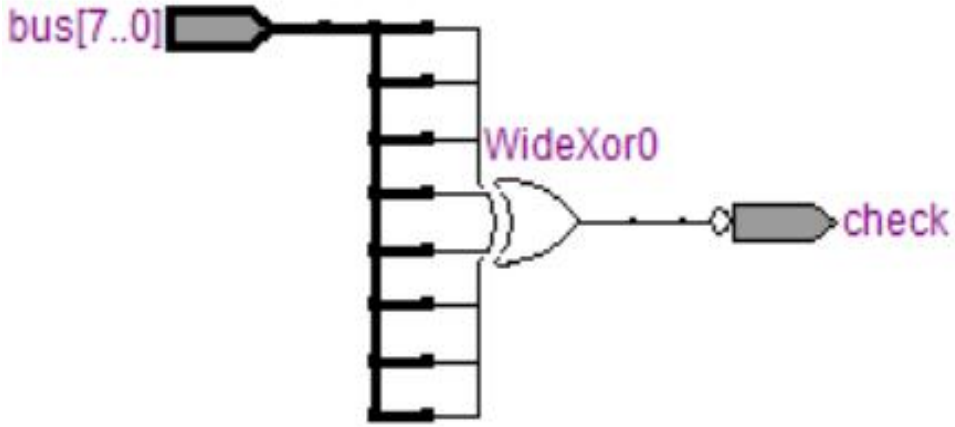
endmodule
```

奇偶？

```
assign  check  =~ ^bus ;
```







二、 时序逻辑电路设计

锁存器、寄存器、计数器、分频器、节拍发生器、状态机等。

时钟上升沿：

```
always @ (posedge clk)
```

时钟下降沿：

```
always @ (negedge clk)
```

原则1：边沿触发信号,不能在always过程中再次出现；

原则2：电平触发信号（或异步控制信号），在always过程中必须用if条件语句对敏感信号表中的信号有匹配的表述，明示信号的逻辑行为。

原则3：同步控制信号，不能出现在敏感信号表中;

原则4：敏感信号列表中不能同时出现边沿触发和电平触发;

原则5：异步敏感信号，异步控制信号在敏感信号表里是边沿敏感信号，但电路性能上是电平敏感;

原则6：尽量采用同步时序 。

1. 同步复位D触发器(寄存器)

敏感信号表中只有时钟信号——同步复位。

```
module dff ( clk, d , reset , q, q_not  );
 input clk, d , reset;
 output q , q_not ;
 reg qtemp ;

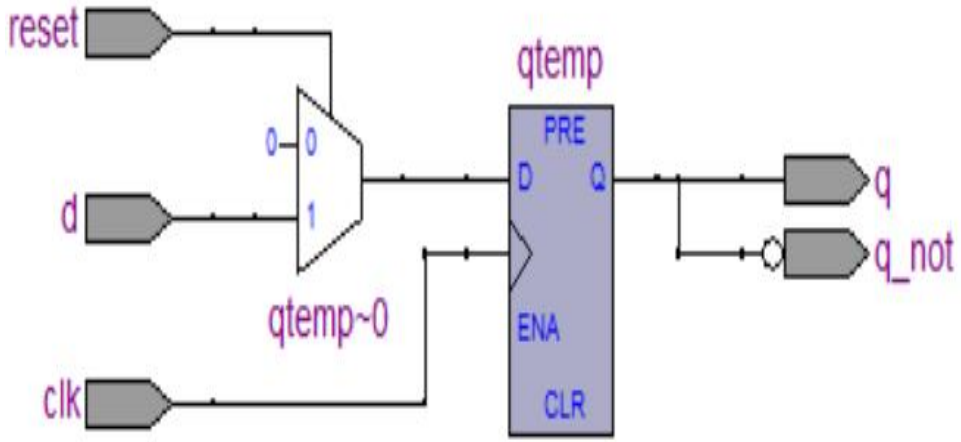
 always @ ( posedge clk )
   begin
    if ( !reset )
       qtemp <= 1'b0 ;
    else
       qtemp <= d ;
   end

  assign q = qtemp ;
  assign q_not = ~qtemp ;

endmodule
```

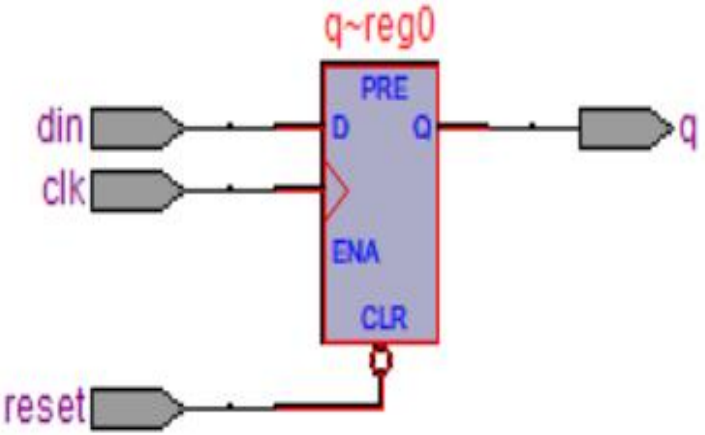
先检查同步信号

if …else都对应时钟沿



2. 异步复位D触发器

敏感信号表中除时钟外，还有其它信号——异步操作




```
always @ ( posedge clk , negedge reset )
   begin
       if ( !reset )
           q <=1'b0;
       else
           q <= din ;
   end
```

- 异步信号必须放在敏感信号表中；
- 必须都是边沿；

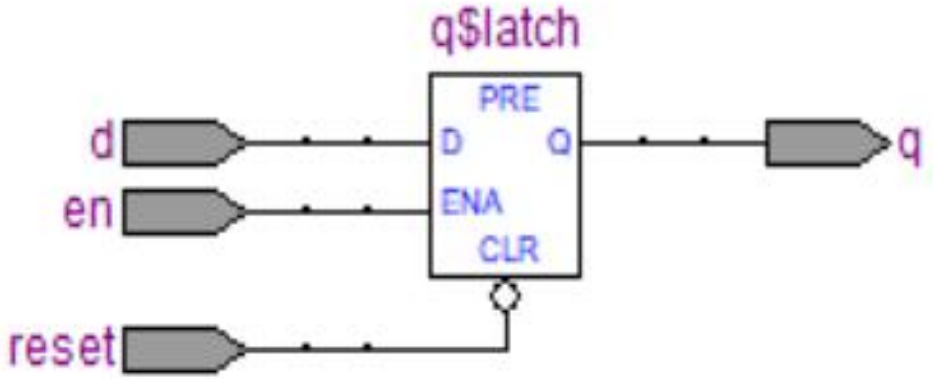
先if描述异步

else分支对应时钟沿

同步、异步区别：敏感信号表

3. 带复位的锁存器

```
always @ ( en , d, reset )
   begin
       if ( !reset )
           q <= 1'b0 ;
       else if ( en )
           q <= d ;
       else
           q <= q ;
  end
```



- 锁存器的所有输入都放在敏感信号表中；
- 锁存器敏感信号都是电平。

不建议综合成锁存器。

4. 计数器

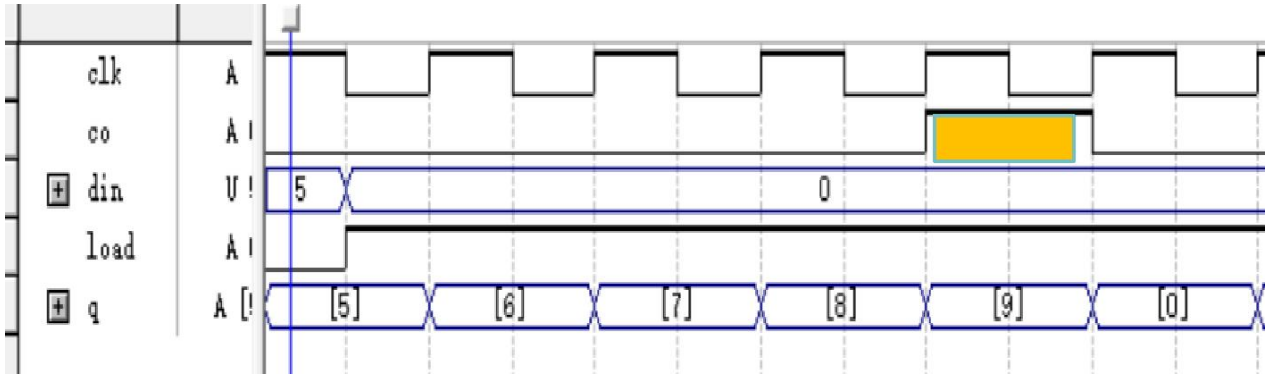
例：同步置数(低有效）的10进制计数器。

```
module count10 ( clk, load , din , q, co );
input clk , load ;
input [3:0] din ;
output [3:0]  q ;
output co ;
reg [3:0]  qtemp ;

always @ ( posedge clk )
    begin
        if (!load)
              qtemp <= din;
        else if  ( qtemp == 4'd9 )
              qtemp <= 4'd0 ;
        else
              qtemp <= qtemp+4’b1 ;
     end

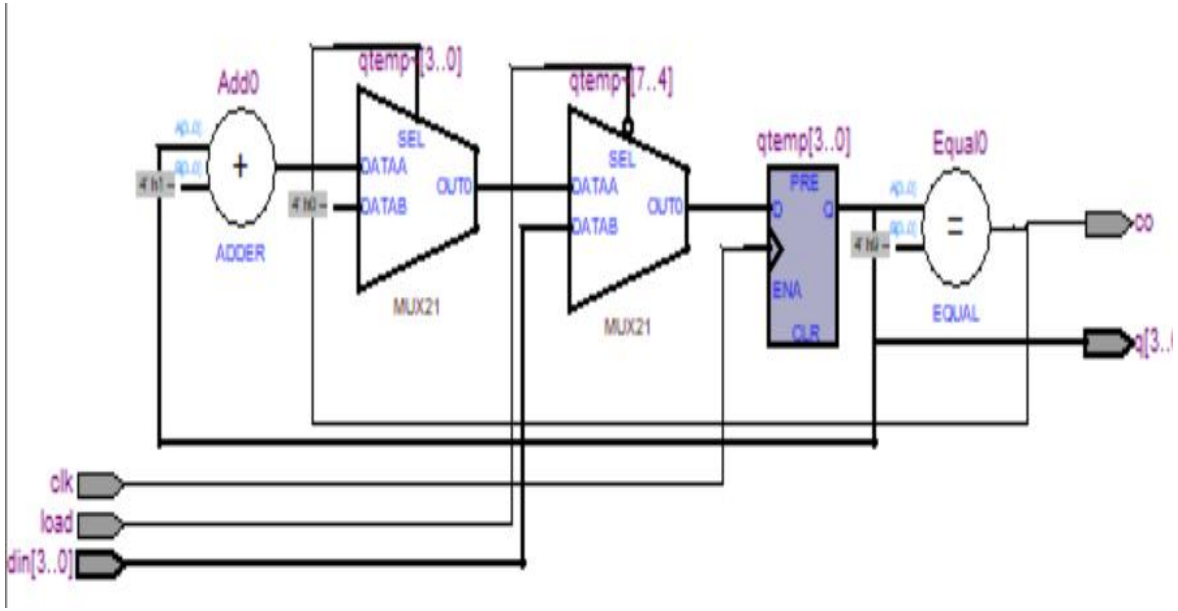
 assign q = qtemp ;
 assign co = (qtemp == 4'd9)  ;

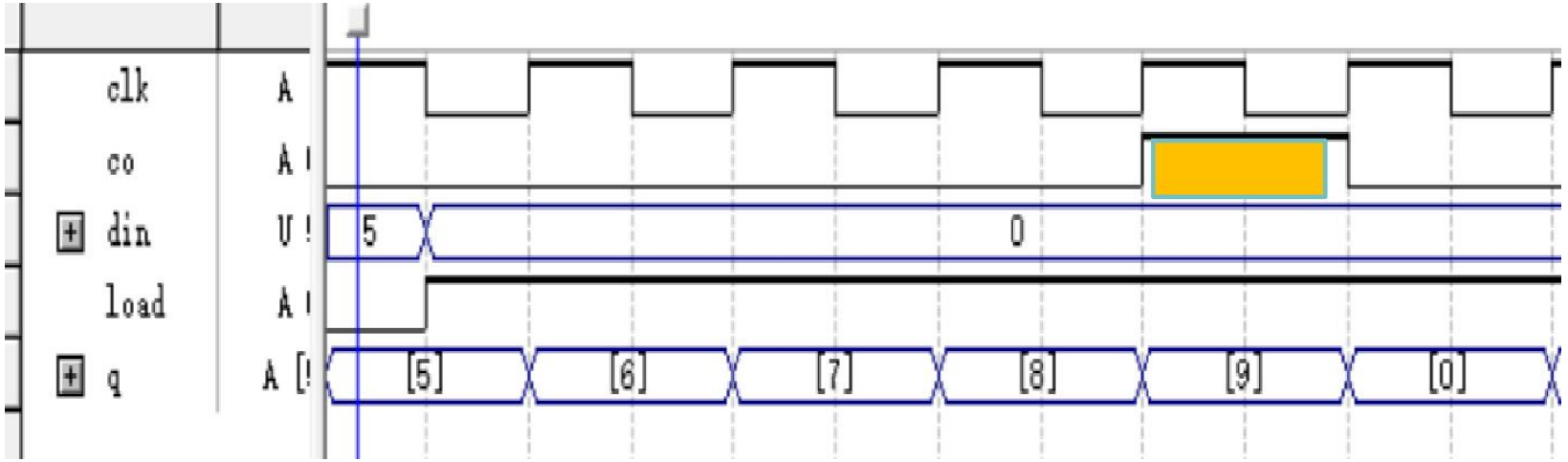
endmodule
```



```
always ( posedge clk )
   begin
      if ( qtemp == n'dm-1 )
            qtemp <= n'd0 ;
       else
            qtemp <= qtemp+n'b1;
   end

assign q = qtemp;
assign c = (qtemp == n'dm-1)  ;
```





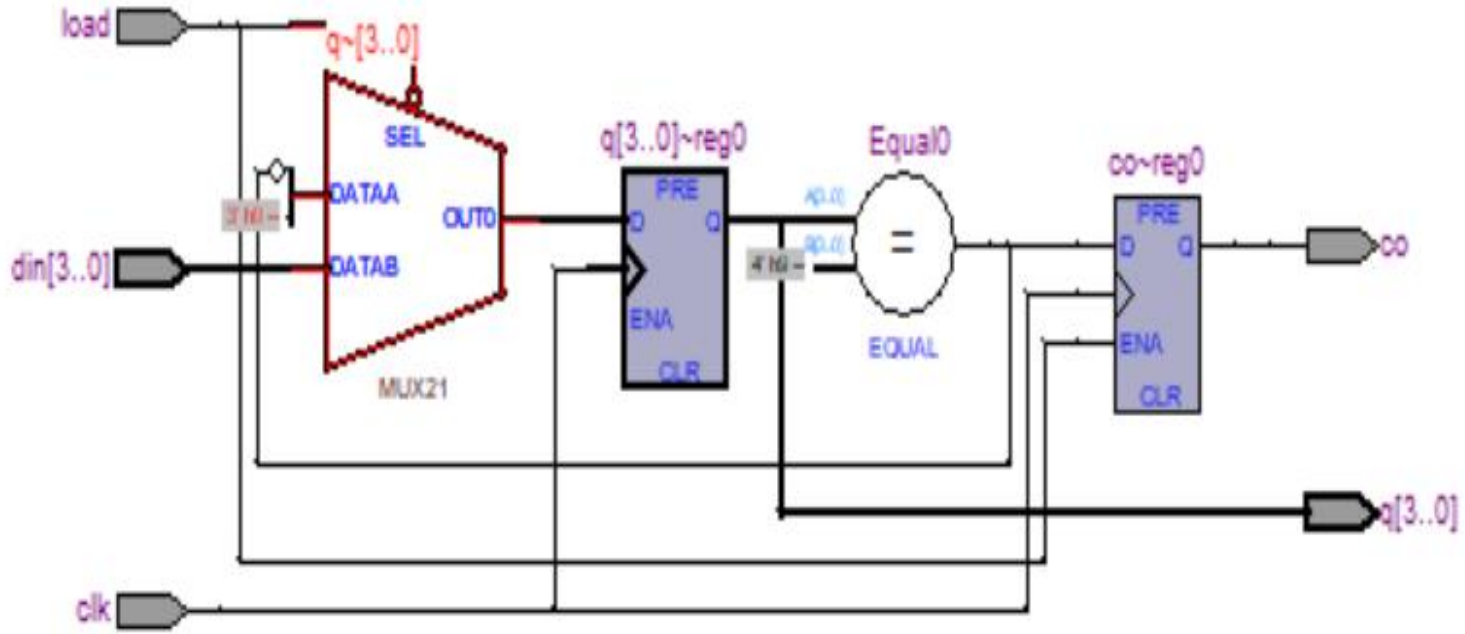
```
module count10(clk, load , din, q, co  );

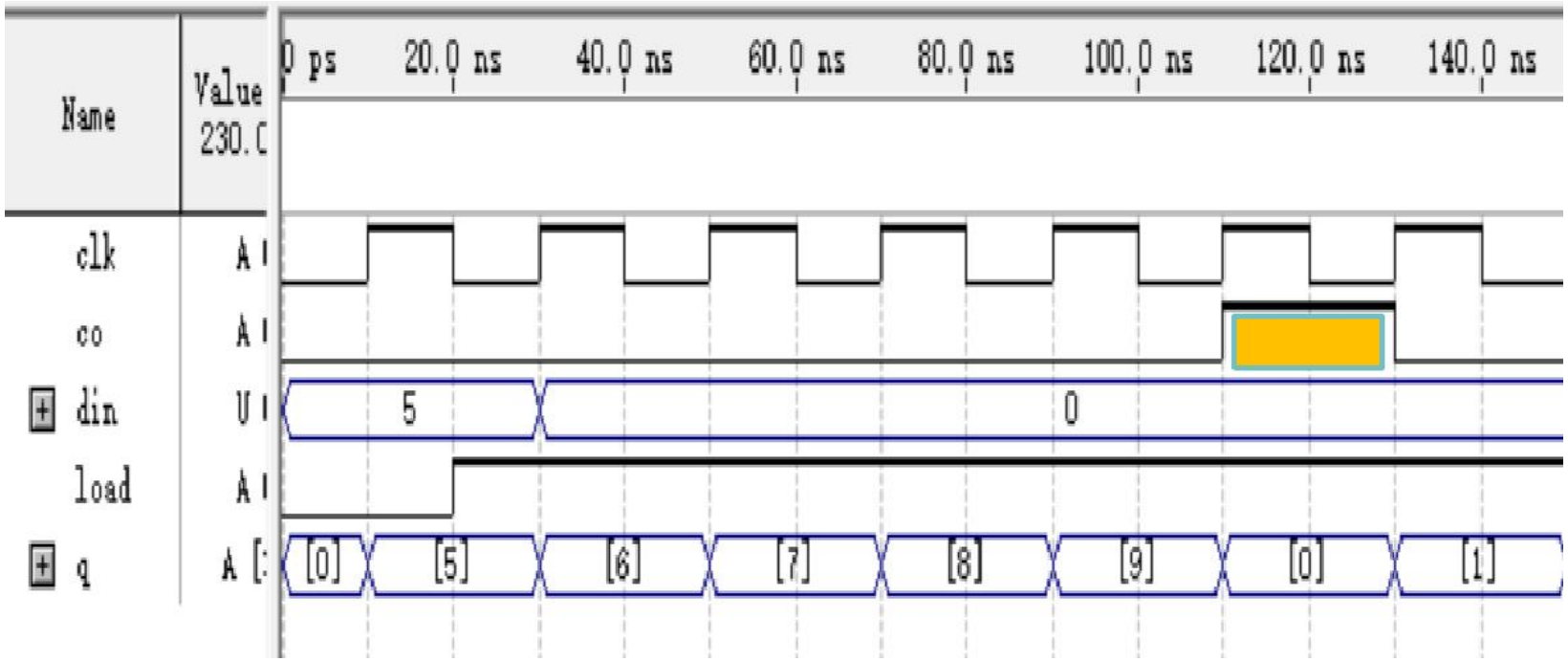
input clk , load ;
input [3:0] din ;
output [3:0]  q ;
output co ;
reg [3:0]  q ;
reg co ;

always @ (posedge clk)
   begin
      if (!load)
            q <= din;
```

```
  else if  ( q == 4'd9 )
        begin
            q <= 4'd0 ;
            co <= 1'b1 ;
         end
   else
        begin
            q <= q+4’b1 ;
            co <= 1'b0 ;
        end
    end

endmodule
```


load
q~[3..0]
SEL
DATAA
DATAB
OUT0
din[3..0]
MUX21
q[3..0]~reg0
PRE
D
Q
ENA
CLR
Equal0
=
EQUAL
co~reg0
co
q[3..0]
clk


Name
Value
230.0
0 ps
20.0 ns
40.0 ns
60.0 ns
80.0 ns
100.0 ns
120.0 ns
140.0 ns
clk
co
din
5
0
load
q
[0]
[5]
[6]
[7]
[8]
[9]
[0]
[1]

```
module count161 ( clk ,EP ,ET, cr , ld , din , qout ,co );
  input clk ,EP ,ET , cr , ld ;
  input [3:0] din;
  output [3:0] qout ;
  output co ;
  reg [3:0] qtemp;

always @(posedge clk , negedge cr )
  begin
    if (!cr)
      qtemp <= 4'b0000 ;
    else if ( !ld )
      qtemp <= din ;
    else if ( ET && EP )
      qtemp <= qtemp + 1'b1 ;
    else qtemp <= qtemp ;
  end
```

例：74LS161。

输入									输出					CT74LS161
$\overline{CR}$	$\overline{LD}$	EP	ET	CP	D_3	D_2	D_1	D_0	Q_3	Q_2	Q_1	Q_0	CO	
0	×	×	×	×	×	×	×	×	0	0	0	0	0	异步置 0
1	0	×	×	↑	d_3	d_2	d_1	d_0	d_3	d_2	d_1	d_0		$CO = E_T \cdot Q_3Q_2Q_1Q_0$
1	1	1	1	↑	×	×	×	×	计	数				$CO = Q_3Q_2Q_1Q_0$
1	1	0	×	×	×	×	×	×	保	持				$CO = E_T \cdot Q_3Q_2Q_1Q_0$
1	1	×	0	×	×	×	×	×	保	持			0	

```
  assign co = ET & (qtemp== 4'd15) ;
  assign qout = qtemp ;

endmodule
```

例：异步清零的24进制（小时）计数器

与二进制计数器不同，分别做个位、十位计数

```
module count_24 ( clk , clr, s1 , s10, co );

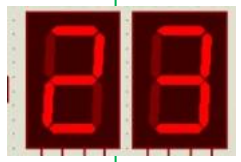
input clk , clr ;
output [3:0]  s1, s10 ;
output co ;
reg [3:0]  s1tmp , s10tmp ;

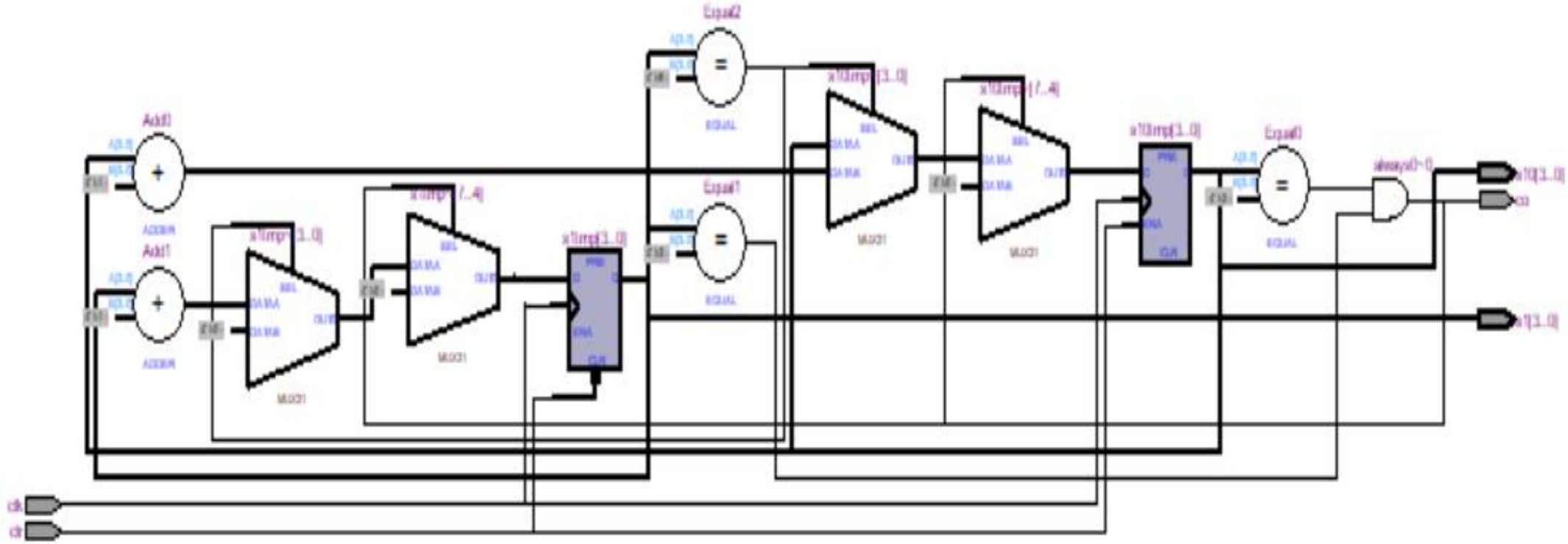
always @ ( posedge clk , negedge clr )
   begin
    if ( !clr )
      begin
        s1tmp <= 4'd0 ;
       s10tmp <= 4'd0 ;
      end
```

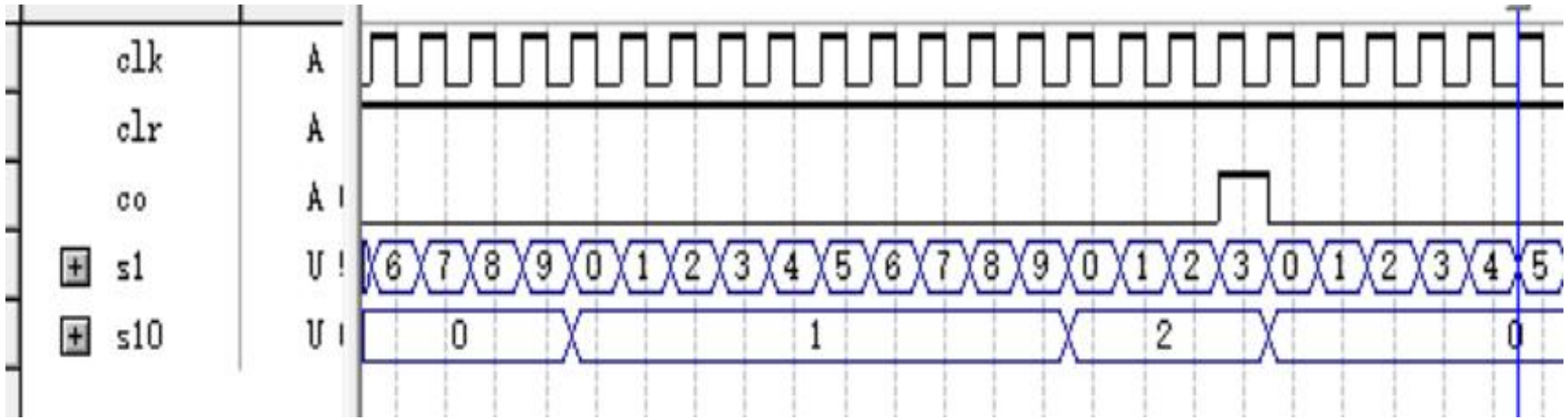
```
  else if (   s10tmp == 4'd2 && s1tmp == 4'd3   )
      begin
           s10tmp <= 4'd0 ;
           s1tmp <= 4'd0 ;
      end
  else if ( s1tmp == 4'd9 )
      begin
         s1tmp <= 4'd0 ;
         s10tmp <= s10tmp + 4'd1 ;
       end
   else
         s1tmp <= s1tmp + 4'd1 ;
  end

assign s1 = s1tmp ;
assign s10 = s10tmp ;
assign co = (s10tmp==4'd2) && ( s1tmp==4'd3) ;

endmodule
```






clk
clr
co
s1
s10
A
A
A
U
U
6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 0 1 2 3 4 5
0
1
2
0

模可控计数器（由输入x, y控制）

M=3,4,6,7

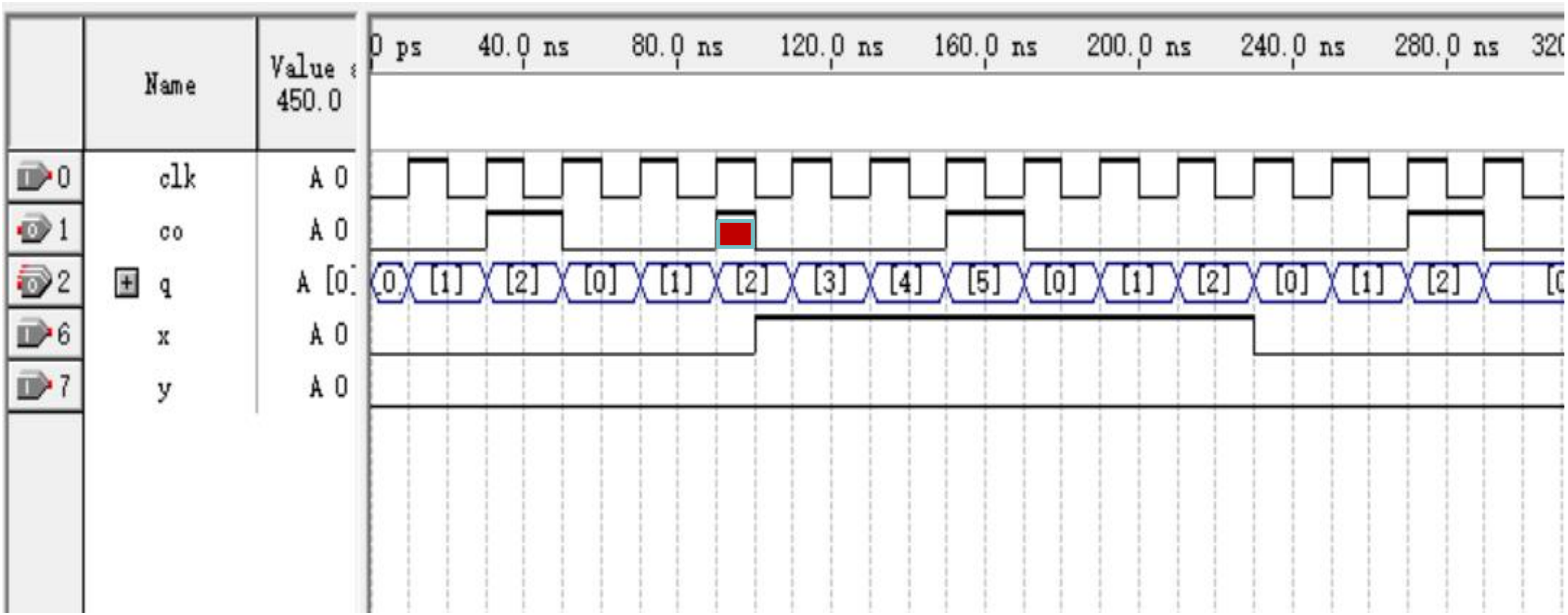
```
module count(  clk , x , y  , q , co );
input clk , x , y ;
output [2:0]  q;
output co ;
reg [2:0]  qtemp , m ;

always @ ( x ,y )       // 组合过程
 begin
    case ({x , y} )
      2'b00:  m = 3'd3 ;
      2'b01:  m = 3'd4 ;
      2'b10:  m = 3'd6 ;
      default: m = 3'd7 ;
    endcase
 end
```

```
always @ ( posedge clk  )   // 时序过程
 begin
     if ( qtemp == m-3'd1 )
        qtemp <= 3'd0 ;
     else
        qtemp <= qtemp + 3'd1
 end

 assign q = qtemp ;
 assign co = (qtemp == m-3'd1)   ;

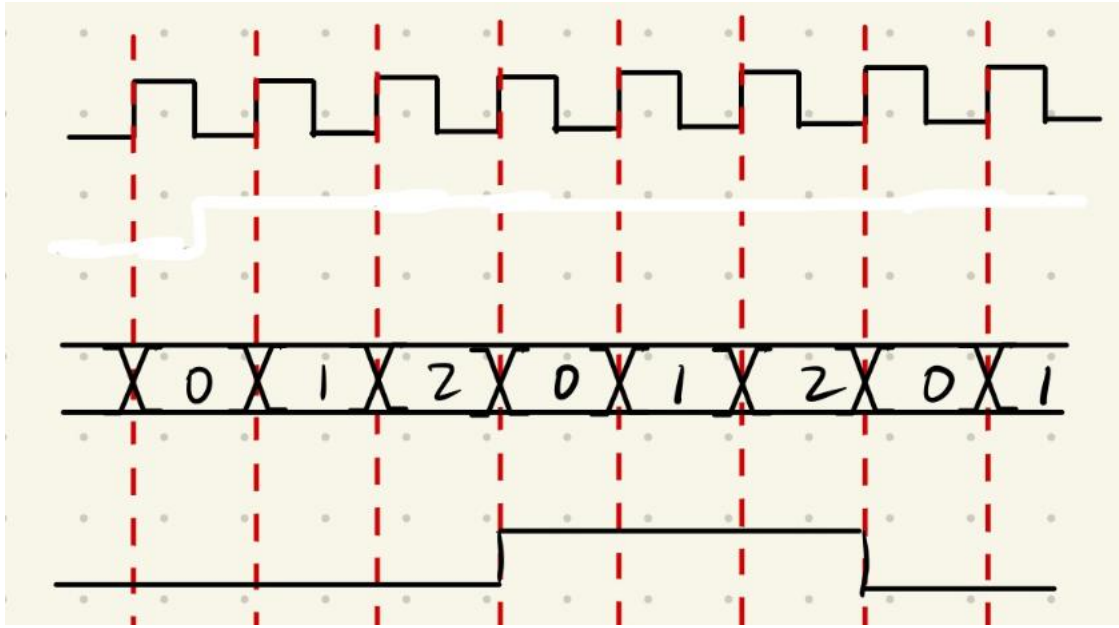
endmodule
```

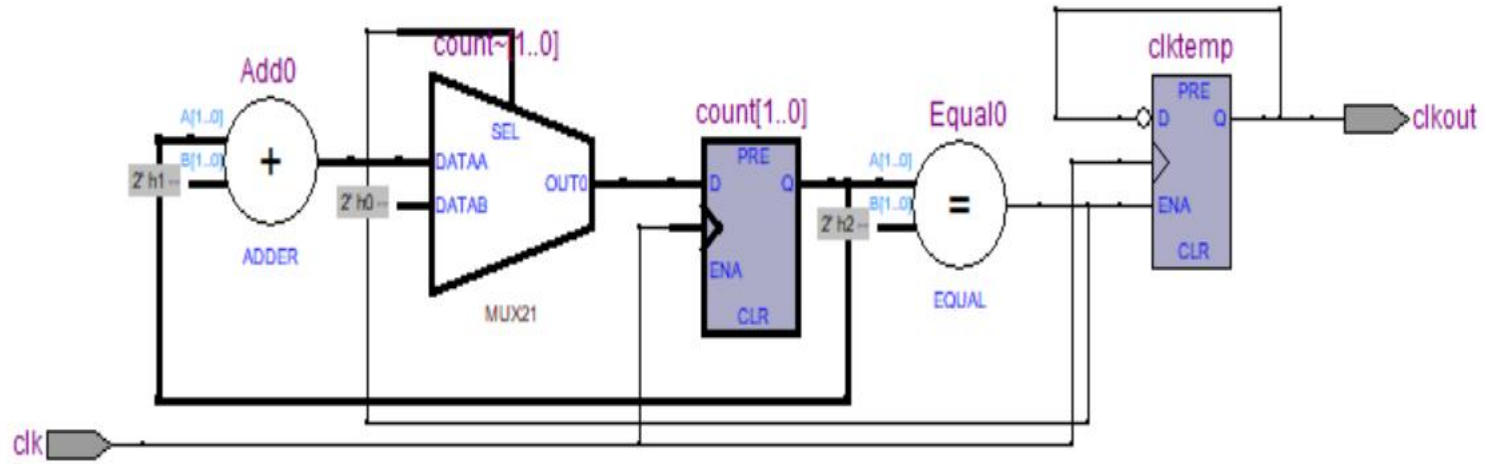

Name
Value
450.0
0 ps
40.0 ns
80.0 ns
120.0 ns
160.0 ns
200.0 ns
240.0 ns
280.0 ns
0
clk
A 0
1
co
A 0
2
q
A [0
[0] [1] [2] [0] [1] [2] [3] [4] [5] [0] [1] [2] [0] [1] [2]
6
x
A 0
7
y
A 0

```
module devider ( clk , clkout   );
input clk  ;
output   clkout ;
reg [1:0]  count ;
reg clktemp;
always @ ( posedge clk )
 begin
    if ( count == 2'd2 )
       count <= 2'd0 ;
    else
       count <= count+2'd1 ;
 end

always @ ( posedge clk  )
 begin
     if (count == 2'd2 )
         clktemp <= ~clktemp ;
     else
         clktemp <= clktemp;
  end
assign clkout = clkoutemp ;
  endmodule
```

5. 设计6分频器，占空比50%





5. 设计5分频器，占空比50%

```
module devider (  clk , clkout );
input clk  ;
output   clkout ;
reg [2:0]  count ;
reg temp1 , temp2;

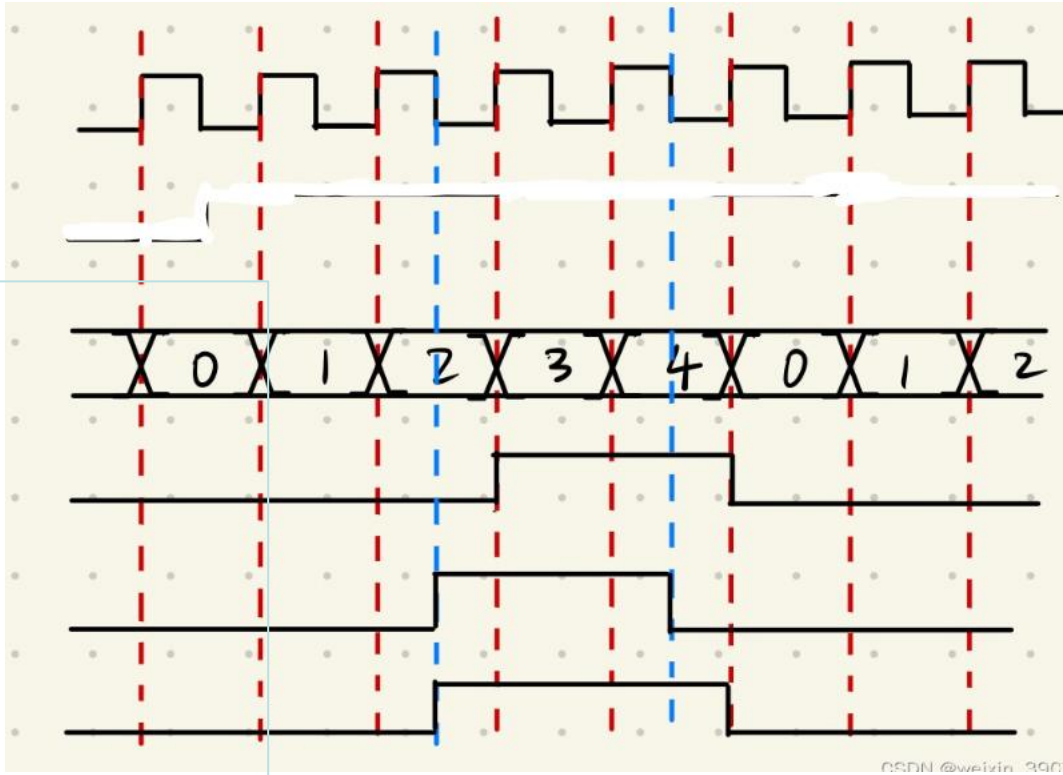
always @ ( posedge clk )
 begin
    if ( count == 3'd4 )
       count <= 3'd0 ;
    else
       count <= count + 3'd1 ;
 end

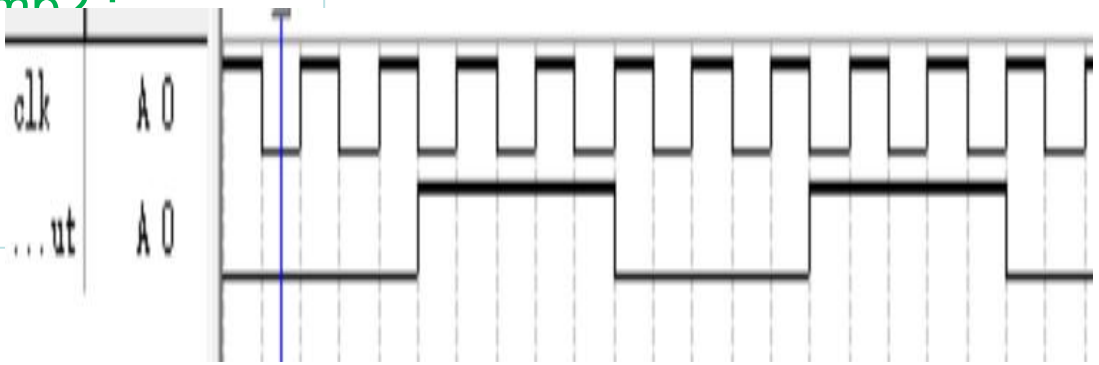
always @ ( posedge clk  )
 begin
     if ( count == 3'd2 )
        temp1 <= 1'b1 ;
     else if ( count == 3'd4 )
        temp1 <= 1'b0 ;
     else  temp1 <= temp1 ;
 end
```

```
always @ (negedge clk )
  begin
      if ( count == 3'd2 )
         temp2 <= 1'b1 ;
      else if ( cout == 3'b4 )
         temp2 <= 1'b0 ;
      else  temp2 <= temp2 ;
  end

assign clkout = temp1 | temp2 ;

endmodule
```





6. 移存型计数器

例：4位环形移位寄存器（左、右移、异步置数1000）

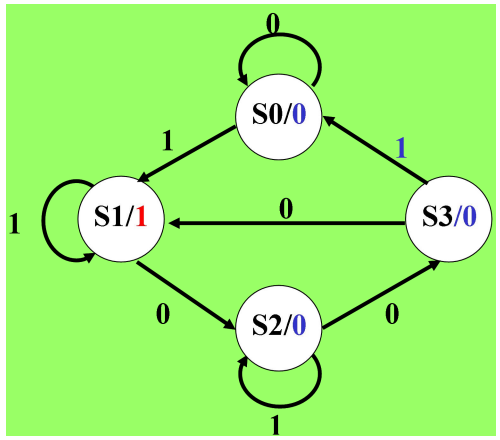
```
module shift ( clk , con , rst , qout  );
input clk , con , rst  ;
output [3:0]  qout ;
reg [3:0]  qtemp;

always @ ( posedge clk , negedge rst )
 begin
    if ( ! rst )
       qtemp <= 4'b1000 ;
    else  if ( con )
            qtemp <= {qtemp[2:0],qtemp[3]} ;       //左移
    else
            qtemp <= {qtemp[0],qtemp[3:1]} ;      // 右移
 end

 assign  qout = qtemp ;

endmodule
```

7. 状态机



1.) 使用两段式描述：

描述时序逻辑（状态改变）

组合逻辑（次态、输出）

2）状态机说明语句：

parameter ： 各状态编码

```
parameter s0=2'b00 , s1=2'b01, s2=2'b10,s3=2'b11 ;

parameter s0=2'b00 , s1=2'b01, s2=2'b11,s3=2'b10 ;

parameter s0=4'b0001 , s1=2'b0010 ;
parameter s2=4'b0100 , s3=2'b1000 ;
```

二进制码（binary）

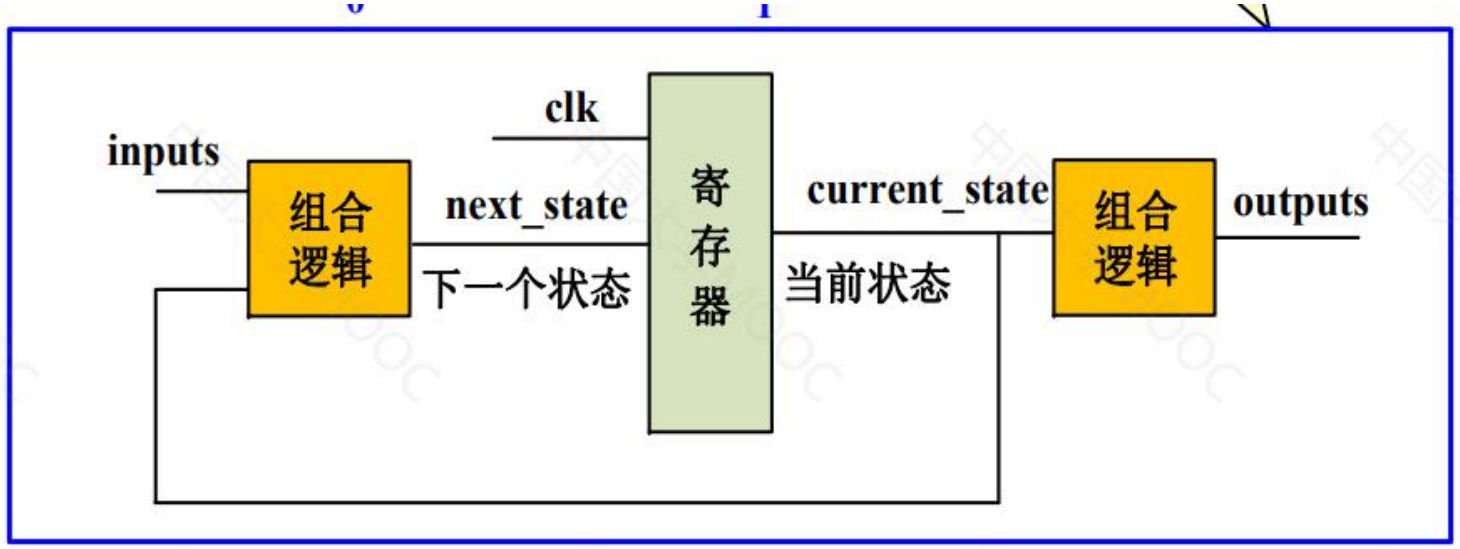
格雷码（Gray）

独热码（one-hot）

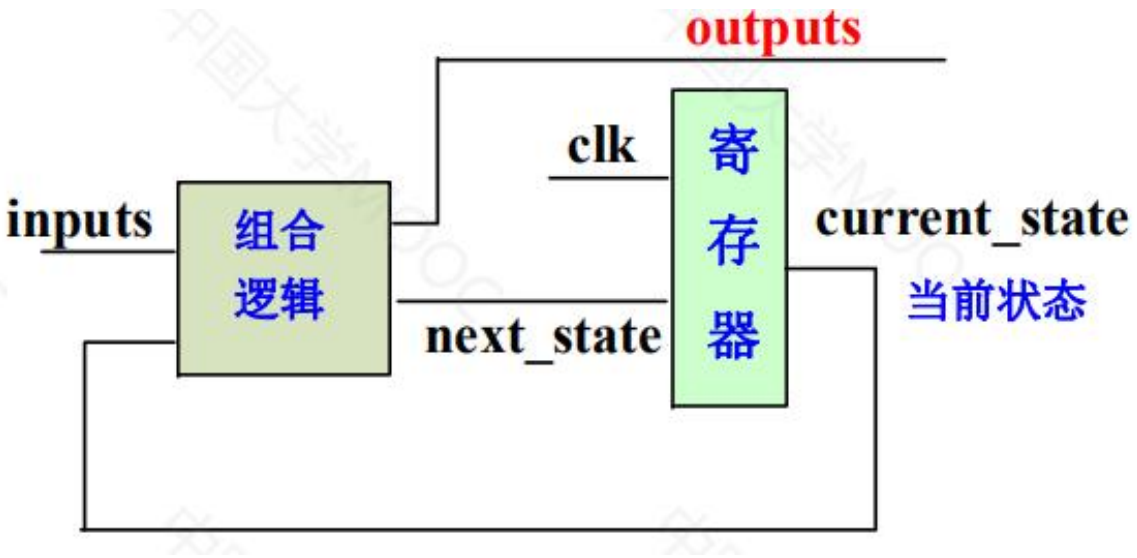
（大型）

3）状态机要复位：避免进入挂起状态。

4）Moore 型状态机:



5）Mealy 型状态机:

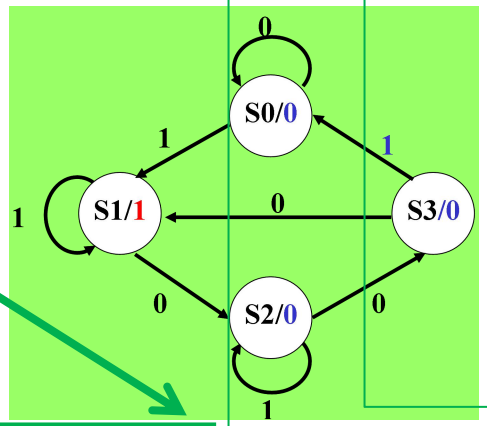


Moore 型状态机的描述

```
module moore ( clk ,x ,reset , y );
input clk , x , reset ;
output   y ;
parameter s0=2'b00 , s1=2'b01 , s2=2'b11 ,
s3=2'b10 ;
reg [1:0]  current_state , next_state;
reg y ;

always @ ( posedge clk , negedge reset ) ;
//时序逻辑
 begin
    if ( ! reset )
       current_state <= s0 ;
    else
        current_state <= next_state ;
 end
```

```
always @ (current_state , x ) ;
 // 组合逻辑
   begin
     case (current_state)
       s0: begin
             y = 1'b0 ;
             if ( x == 1'b0)
                 next_state = s0;
             else
                 next_state = s1;
           end
      s1: begin
             y = 1'b1 ;
             next_state  =  ( x == 1'b0)? s2:s1 ;
           end
```



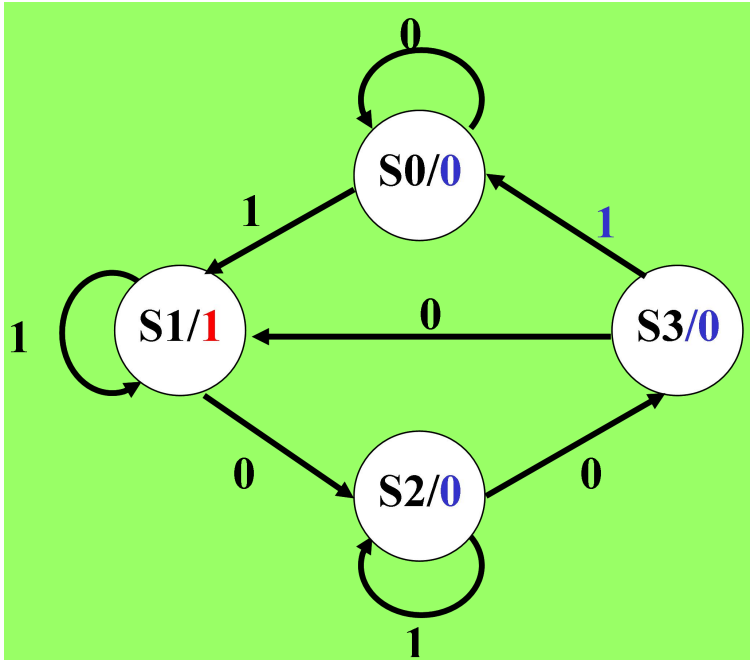
第一进程，完成状态转换，必须饱含时钟敏感信号

第二进程：输出和状态逻辑间关系，必须包含现态、输入等敏感信号

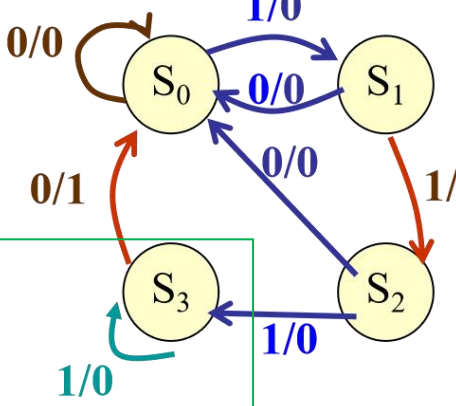
```
        s2: begin
              y  = 1'b0 ;
             next_state = ( x == 1'b0)?  s3:s2 ;
            end
        s3: begin
              y  = 1'b0 ;
             next_state = ( x == 1'b0)?  s1:s0 ;
           end
         default:  begin
                     y = 1'b0 ;
                    next_state = s0;
                  end
      endcase
  end

endmodule
```

摩尔型



Mealy型有限状态机的设计




```
module mealy ( clk ,x ,reset ,  y , );
input clk , x , reset;
output   y ;
parameter s0=2'b00 , s1=2'b01 , s2=2'b11 ,
s3=2'b10 ;
reg [1:0]  current_state , next_state;
reg y ;

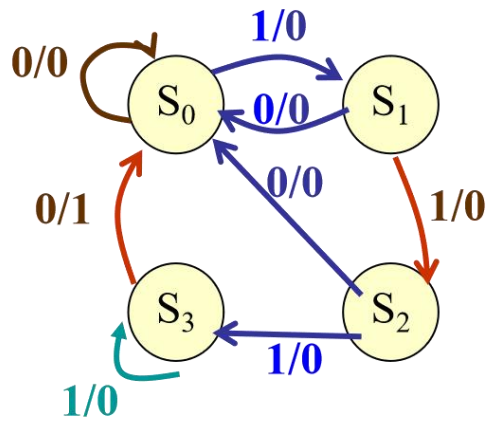
always @ ( posedge clk , negedge reset )
 begin
    if ( ! reset )
       current_state <= s0 ;
    else
        current_state <= next_state ;
 end
```

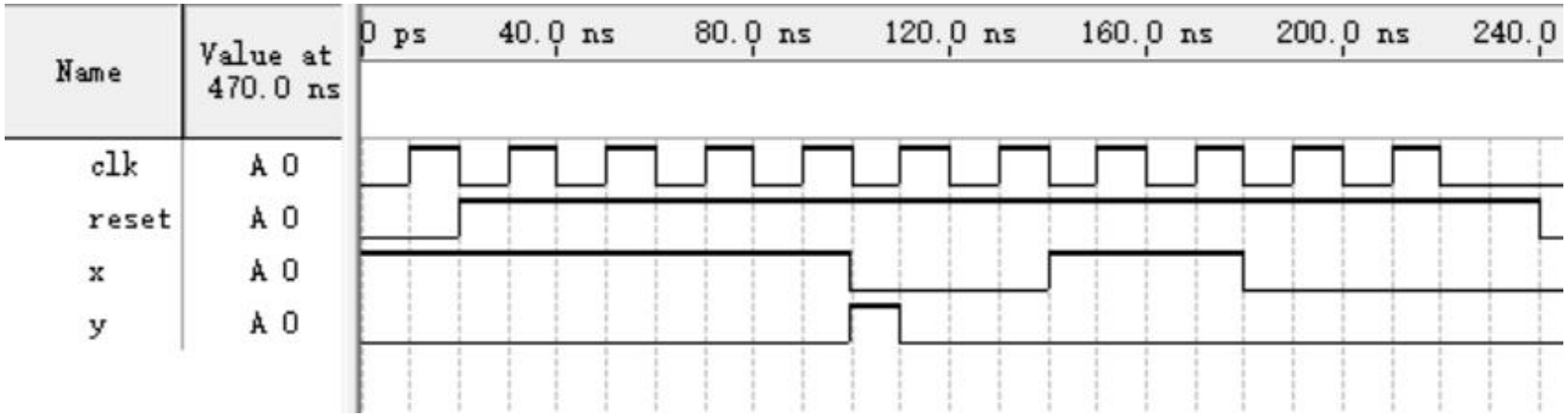
```
always @ (current_state , x)
   begin
     case (current_state)
       s0: begin
             if ( x == 1'b0 )
                begin
                   next_state = s0;
                   y = 1'b0;
                end
             else
                begin
                   next_state = s1;
                   y = 1'b0;
                end
           end
```

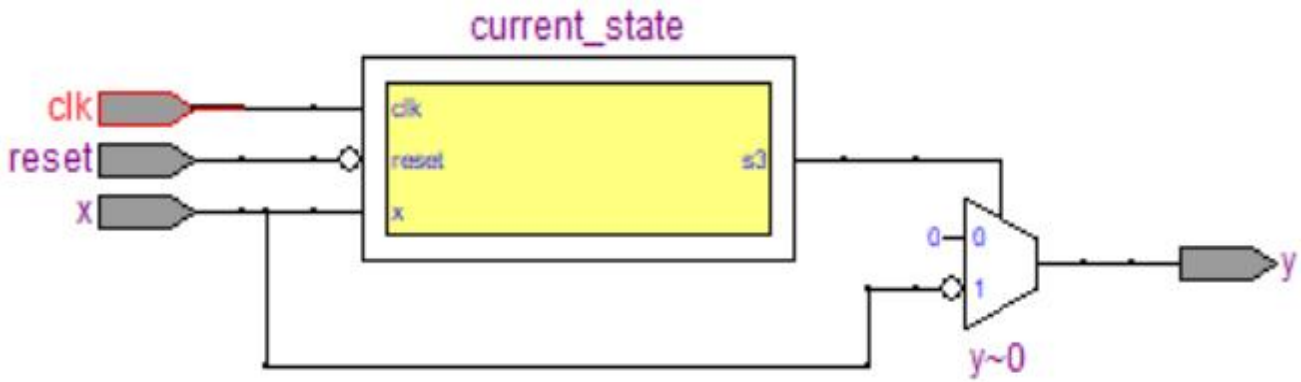
```
s1: begin
        if ( x == 1'b0 )
            begin
              next_state = s0;
              y = 1'b0;
            end
        else
            begin
              next_state = s2;
              y = 1'b0;
            end
    end
s2: begin
        if ( x == 1'b0 )
            begin
              next_state = s0;
              y = 1'b0;
            end
        else
            begin
              next_state = s3;
              y = 1'b0;
            end
    end
```

```
s3: begin
        if ( x == 1'b0 )
            begin
              next_state = s0;
              y = 1'b1;
            end
        else
            begin
              next_state = s3;
              y = 1'b0;
            end
    end
default:  begin
              y = 1'b0 ;
              next_state = s0;
          end
endcase
end

endmodule
```



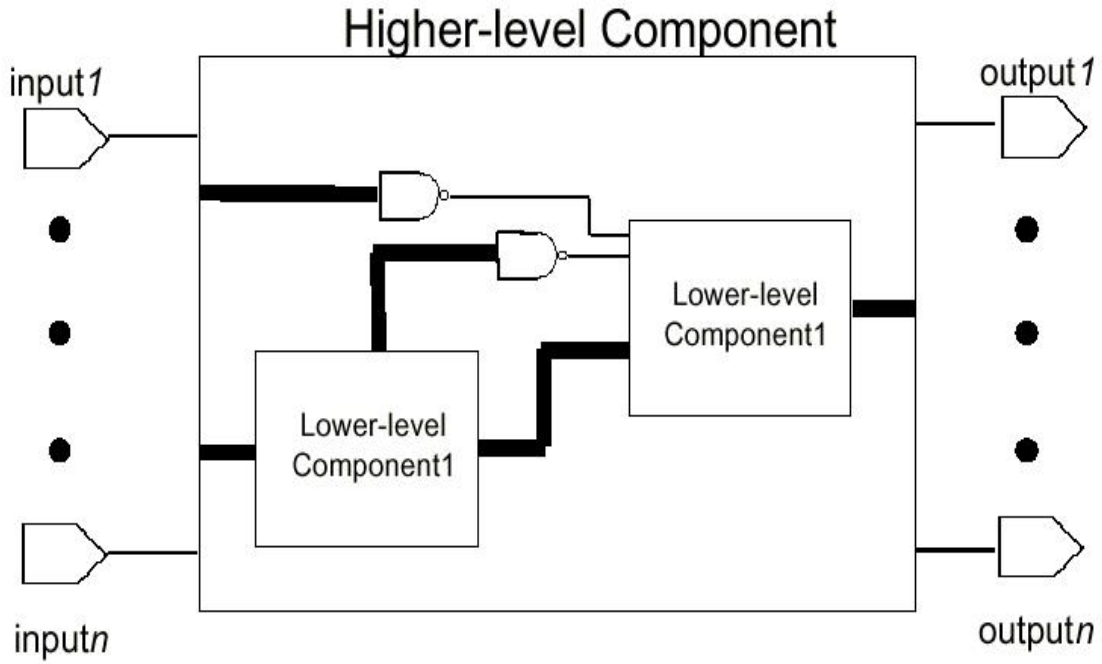

Name
Value at 470.0 ns
0 ps
40.0 ns
80.0 ns
120.0 ns
160.0 ns
200.0 ns
240.0
clk
A 0
reset
A 0
x
A 0
y
A 0


current_state
clk
reset
x
clk
reset
s3
0
0
1
y
y~0

5.5.4 verilog的层次化（结构化）设计

在多层次的设计中，高层次的设计模块调用低层次的设计模块，构成模块化的设计。重点描述模块间的连接关系。

子模块名　例化名　（父子模块端口关联表）；




```
AND  u1 ( a, b, and_out );
AND  u1 ( .a(a),  .b(b),  .o(and_out) );
```

子　父

端口列表：位置关联

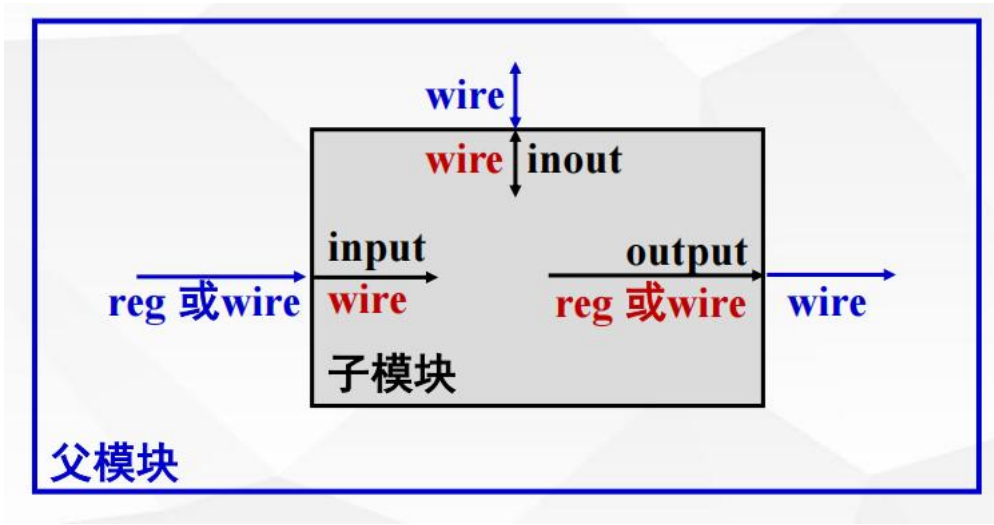
或 名字关联

被例化元件的来源：

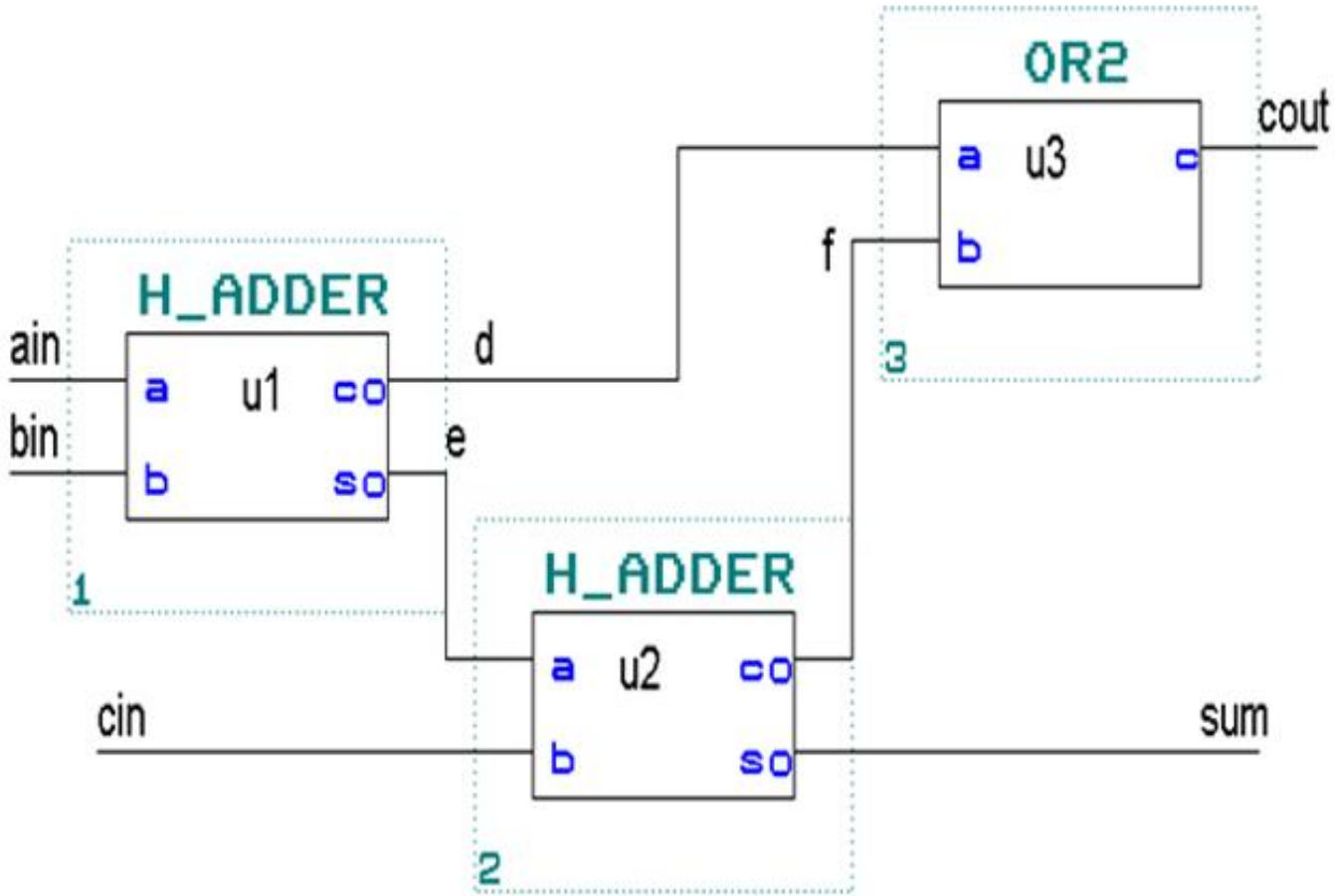
- verilog 设计模块;
- 其它HDL设计实体;
- 厂商提供的工艺库中的元件、IP核。

注意：

- 一个元件是一段结构完整的 module 模块。
- 不能在always语句内部引用子模块。
- 有关模块端口数据类型的规定



一位全加器原理图


OR2
u3
a
b
o
cout
f
3
H_ADDER
u1
ain
bin
a
b
co
so
d
e
1
H_ADDER
u2
cin
a
b
co
so
sum
2

```
module h_adder (  a , b, so ,co  ) ;  //半加器
input a , b;
output so , co ;

assign so = a ^ b ;
assign co = a & b ;

endmodule
```

```
module or_2 ( a , b, c ) ;    //或门
input a , b;
output c ;

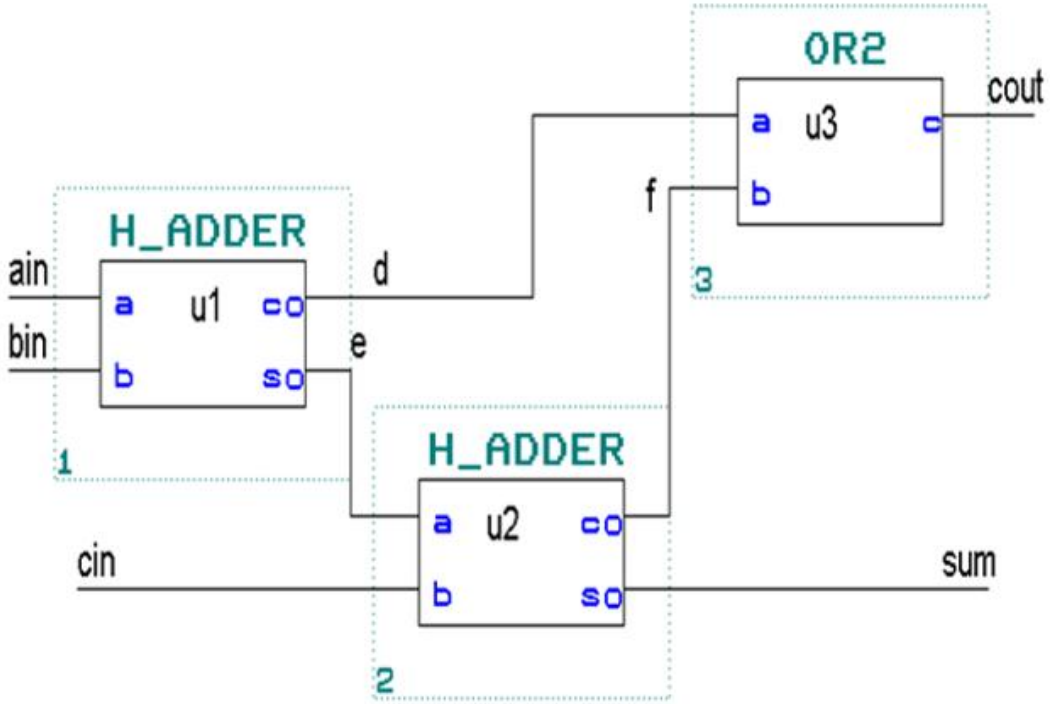
assign c = a | b ;

endmodule
```

```
module adde( ain , bin, cin , sum , cout  );    顶层逻辑
input ain, bin , cin ;
output   sum , cout ;
wire d , e , f ;

h_adder   u1 ( ain , bin, e, d  ) ;   //元件例化
h_adder   u2 (.a(e) , .b(cin) , .so(sum) , .co(f )) ;
or_2   u3 ( d , f , cout ) ;

endmodule
```



```
module lampion(  clk , clr ,y );
input clk , clr ;
output [7:0]   y ;
reg [4:0] count ;
reg [7:0] y ;

always @( posedge clk , negedge clr)
  begin
    if ( !clr )
        count <= 5'd0 ;
     else
        count <= count+5’b1 ;
  end

always @ ( count )
   begin
      case ( count )
       5'd0:  y = 8'b0000_0001 ;
       5'd1:  y = 8'b0000_0010 ;
```

8个彩灯，4花样自动切换的彩灯控制器：

第1：从右到左，然后从左到右逐次点亮，再全黑，再全亮；

第2：两边同时亮1个逐次向中间移动再散开；

第3：两边同时亮2个逐次向中间移动再散开；

第4：每三个亮、每四个亮、间隔亮。

共32个种变换，循环往复。

```
5'd2:  y = 8'b0000_0100 ;
5'd3:  y = 8'b0000_1000 ;
5'd4:  y = 8'b0001_0000 ;
5'd5:  y = 8'b0010_0000 ;
5'd6:  y = 8'b0100_0000 ;
5'd7:  y = 8'b1000_0000 ;
5'd8:  y = 8'b1000_0000 ;
5'd9:  y = 8'b0100_0000 ;
5'd10:  y = 8'b0010_0000 ;
5'd11:  y = 8'b0001_0000 ;
5'd12:  y = 8'b0000_1000 ;
5'd13:  y = 8'b0000_0100 ;
5'd14:  y = 8'b0000_0010 ;
5'd15:  y = 8'b0000_0001 ;
5'd16:  y = 8'b0000_0000 ;
5'd17:  y = 8'b1111_1111 ;
5'd18:  y = 8'b1000_0001 ;
```

```
5'd19:  y = 8'b0100_0010 ;
 5'd20:  y = 8'b0010_0100 ;
 5'd21:  y = 8'b0001_1000 ;
 5'd22:  y = 8'b0010_0100 ;
 5'd23:  y = 8'b0100_0010 ;
 5'd24:  y = 8'b1000_0001 ;
 5'd25:  y = 8'b1100_0011 ;
 5'd26:  y = 8'b0011_1100 ;
 5'd27:  y = 8'b1100_0011 ;
 5'd28:  y = 8'b1110_0111 ;
 5'd29:  y = 8'b0111_1110 ;
 5'd30:  y = 8'b1111_0000 ;
 5'd31:  y = 8'b1111_0000 ;
 default: y = 8'b0000_0000 ;
 endcase
end

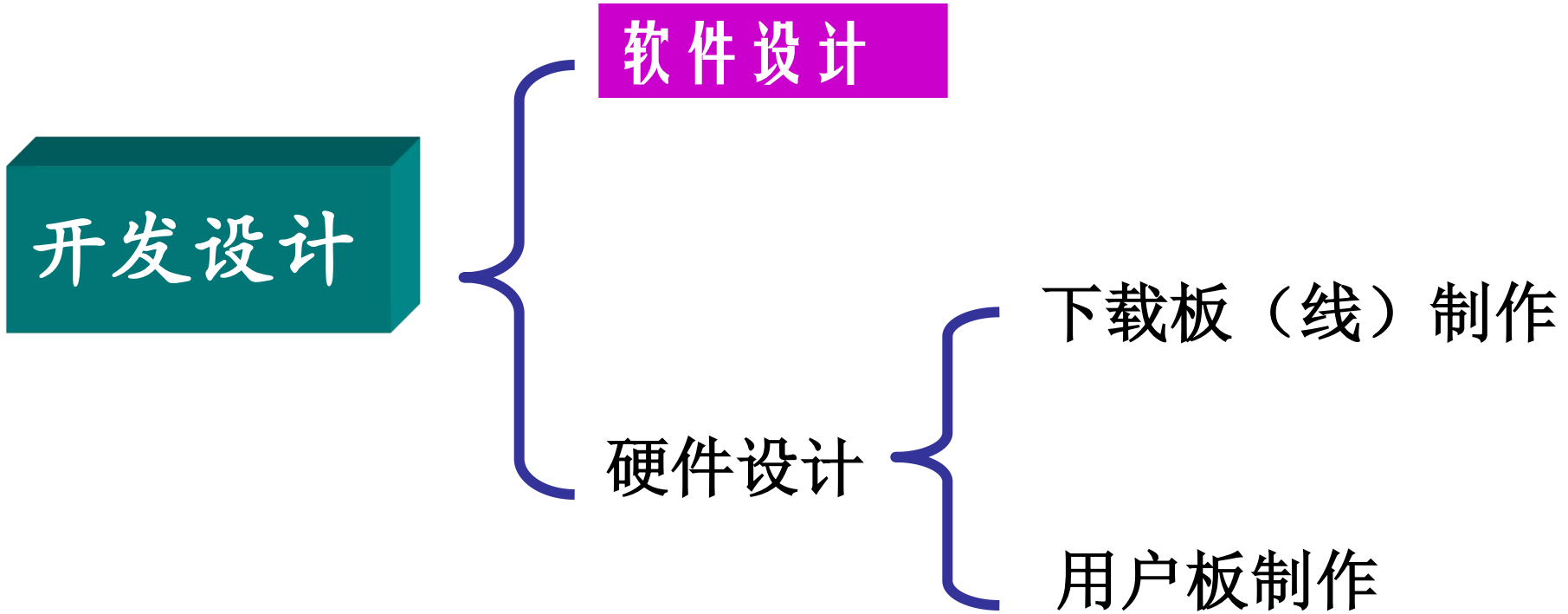
endmodule
```

序列信号发生器如何实现?

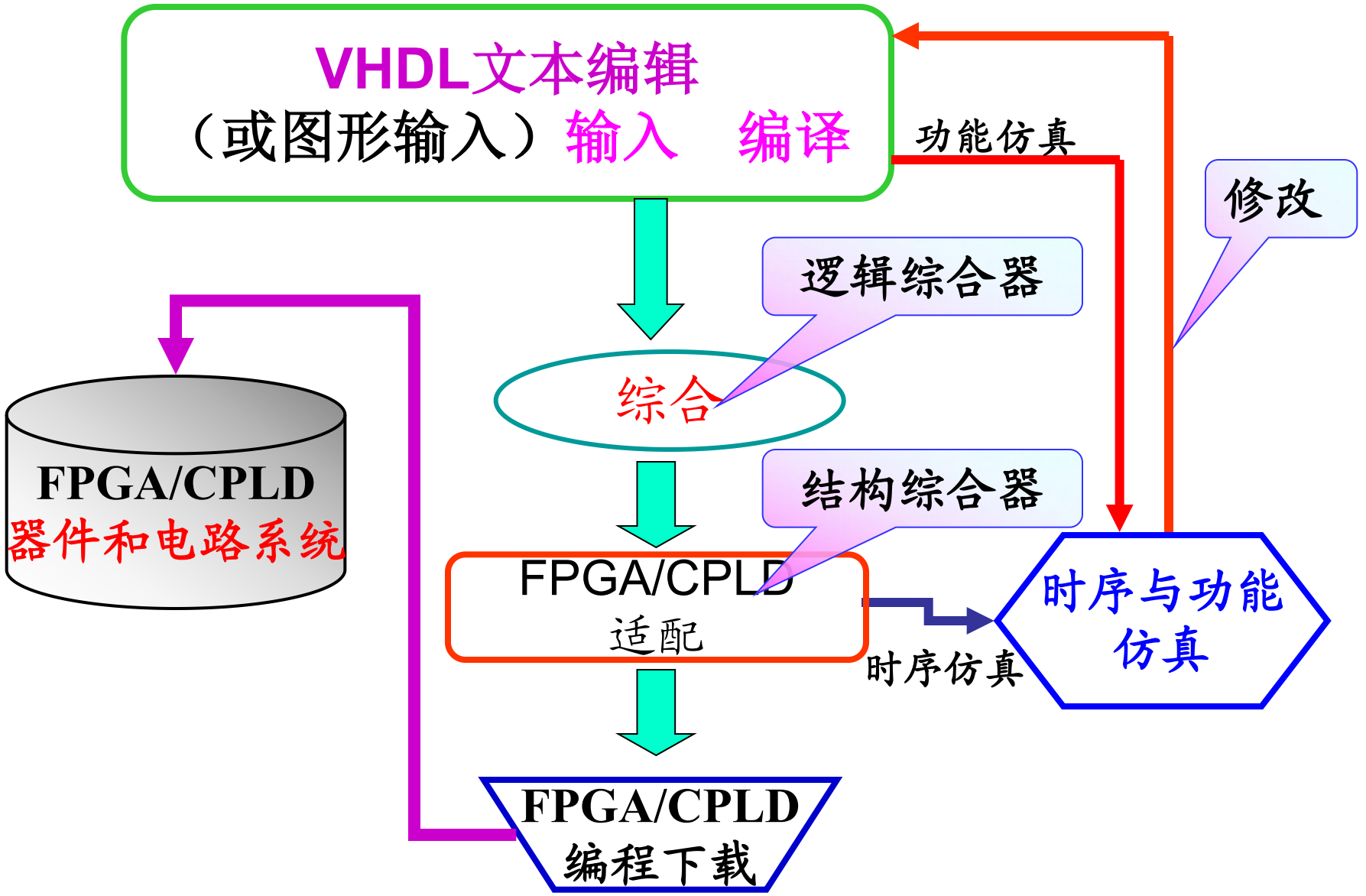
顺序脉冲发生器如何实现?

可编程逻辑器件器件的开发流程

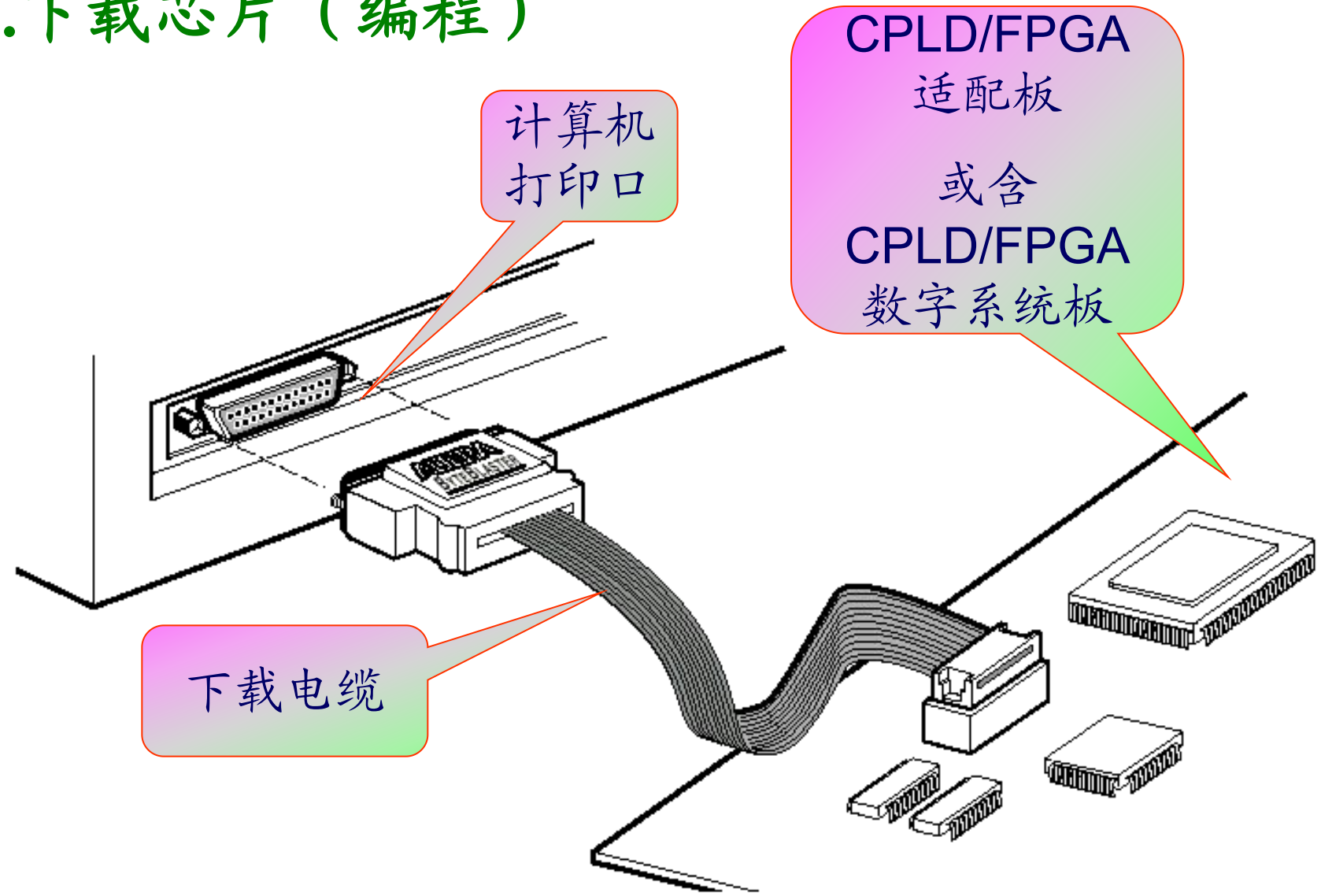
开发工具



三、软件设计流程

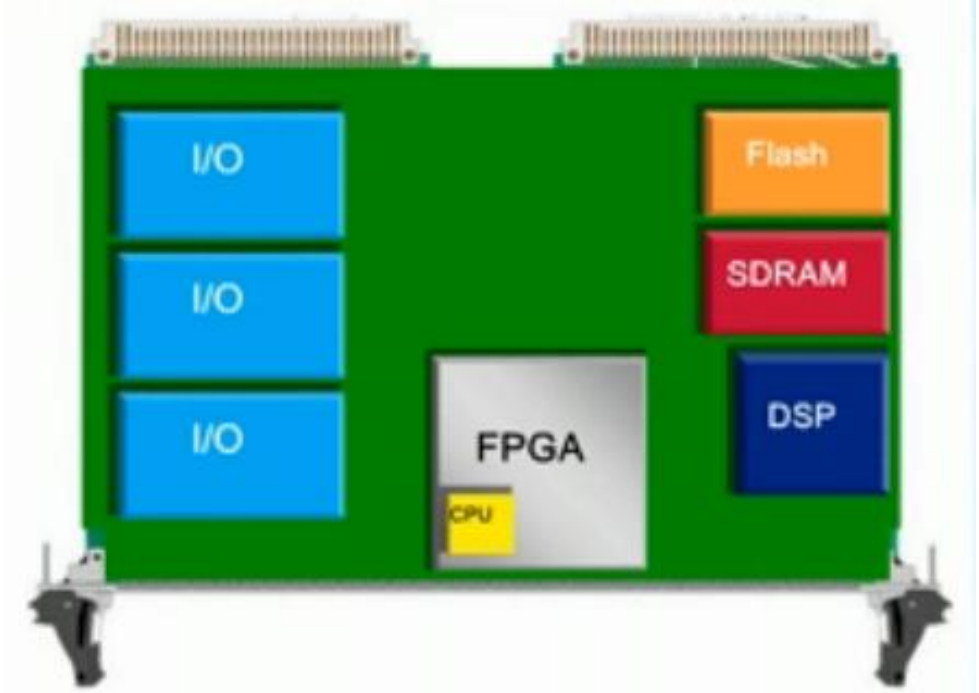

VHDL文本编辑
（或图形输入）输入　编译
功能仿真
修改
逻辑综合器
综合
结构综合器
FPGA/CPLD
器件和电路系统
FPGA/CPLD
适配
时序与功能
仿真
时序仿真
FPGA/CPLD
编程下载

四.下载芯片（编程）



第五章　可编程逻辑器件 PLD

可编程逻辑器件PLD（Programmable logic Device），是由用户自己编程来决定最终逻辑功能和结构的器件。

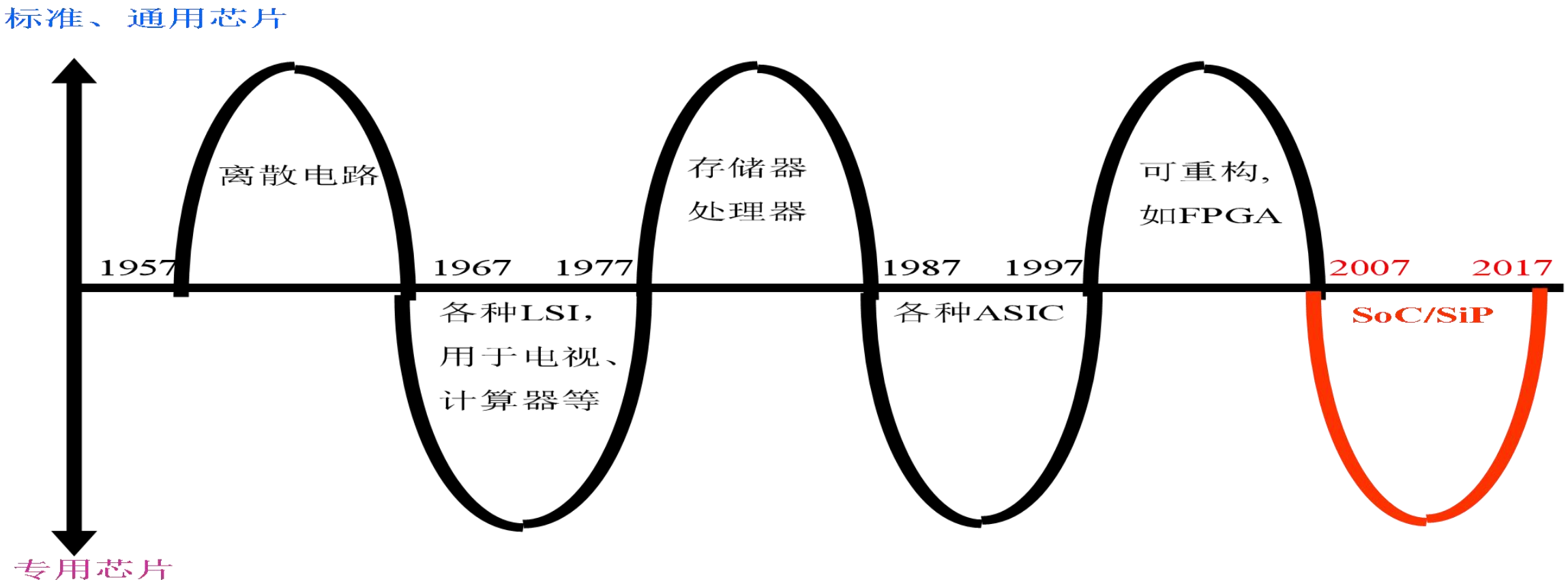


为什么使用PLD？

将这些部件放在一个芯片FPGA中。



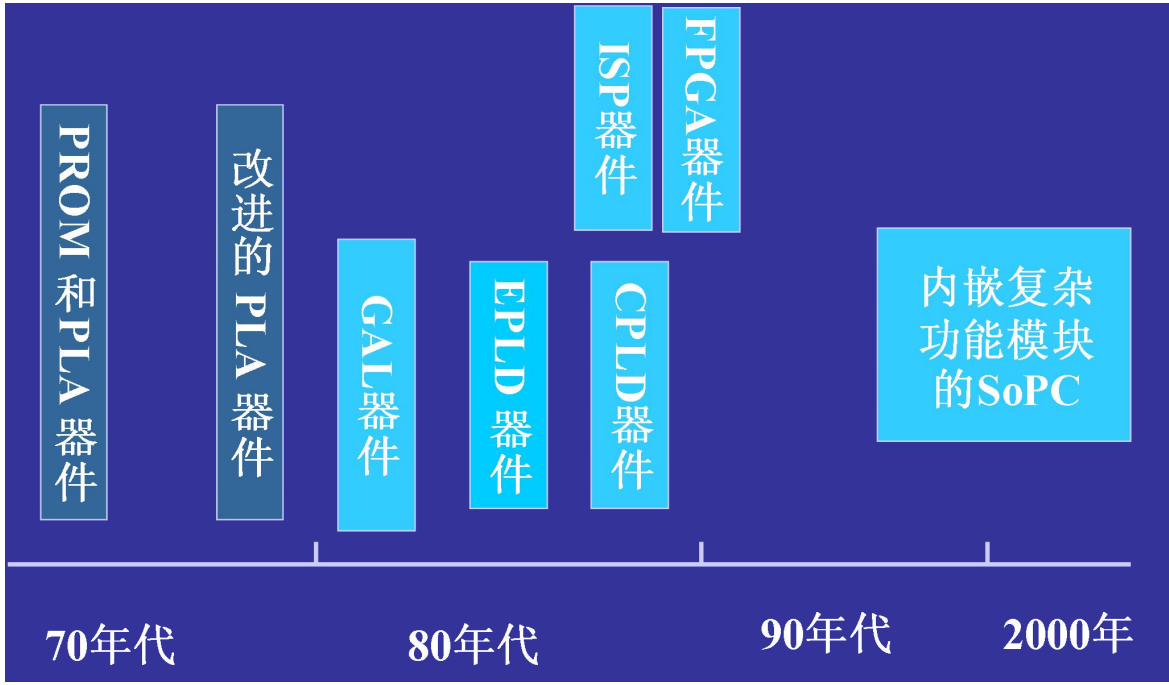
现代数字系统由三种积木块构成：CPU + PLD + RAM



•PLA（Programmable Logic Array）；

•PAL（Programmable Array Logic）；

•GAL（Generic Array Logic）；



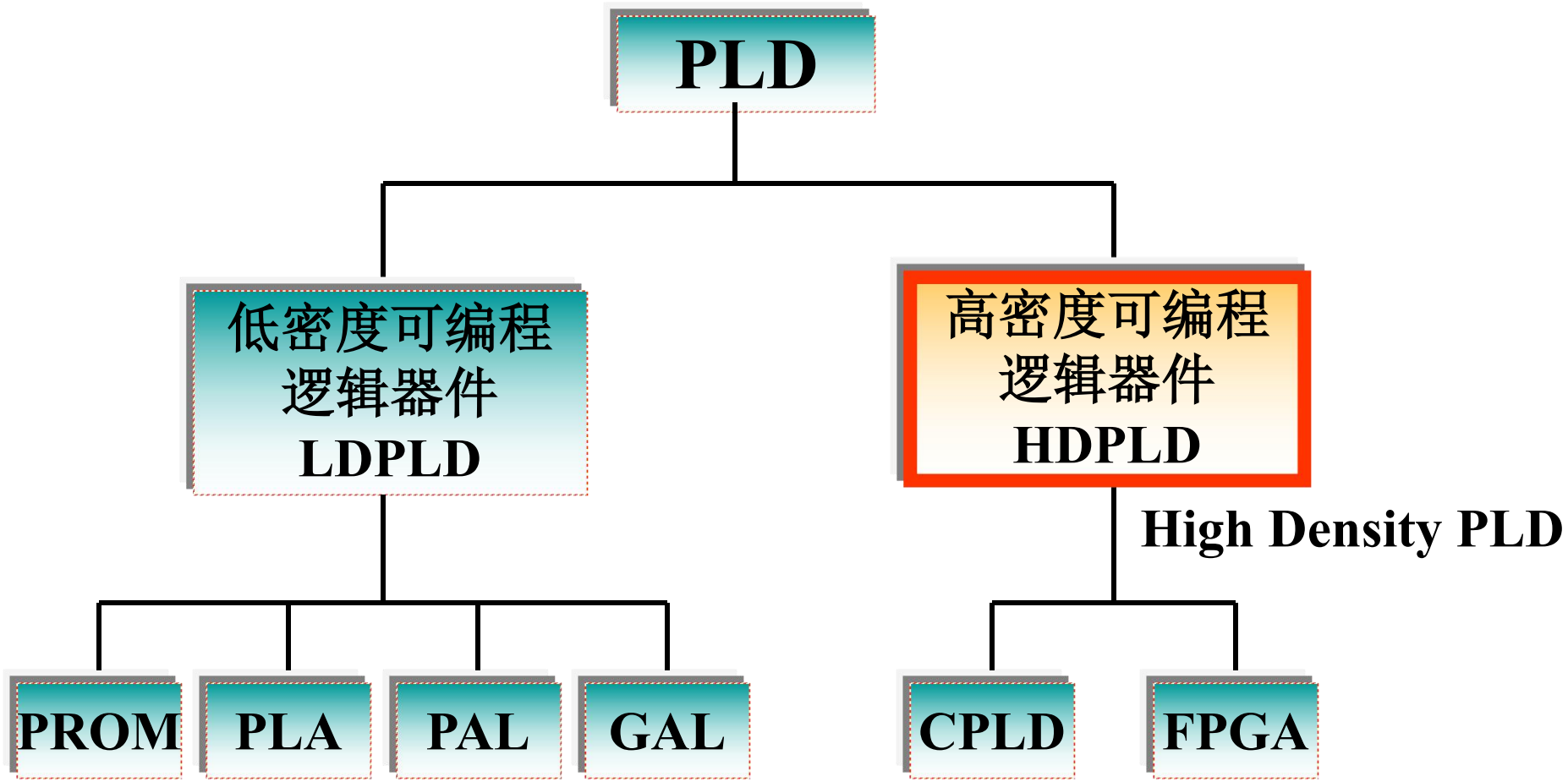
• ISP（In System Porgrammable)；

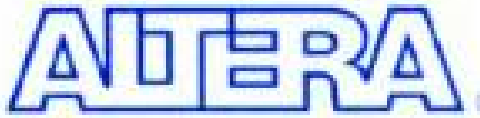
• CPLD （Complex Programmable Logic Device）；

• FPGA（Field Programmable Gates Array）；

•SoPC（System On a Programmable Chip ）

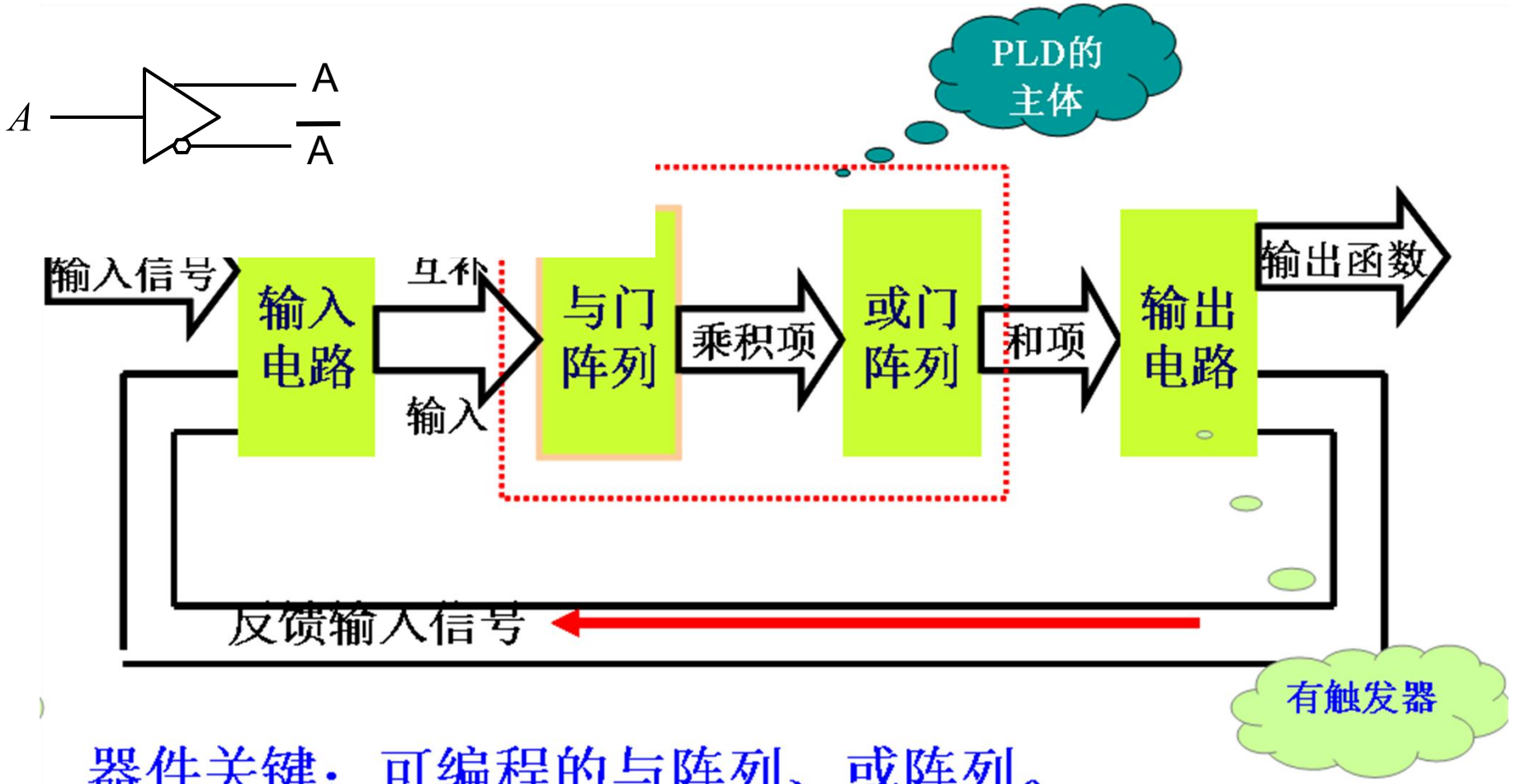
按集成度分类PLD:


PLD
低密度可编程
逻辑器件
LDPLD
高密度可编程
逻辑器件
HDPLD
High Density PLD
PROM
PLA
PAL
GAL
CPLD
FPGA

排名	公司名称	标志	说明
1	XILIXN		FPGA 的发明者
2	ALTERA		
3	Lattice		ISP 技术的发明者
4	ACTEL		反熔丝（一次性烧写）PLD 的领导者
5	Cypress		
6	Quicklogic		
7	Lucent		
8	ATMEl		

§5.1 PLD的基本概念

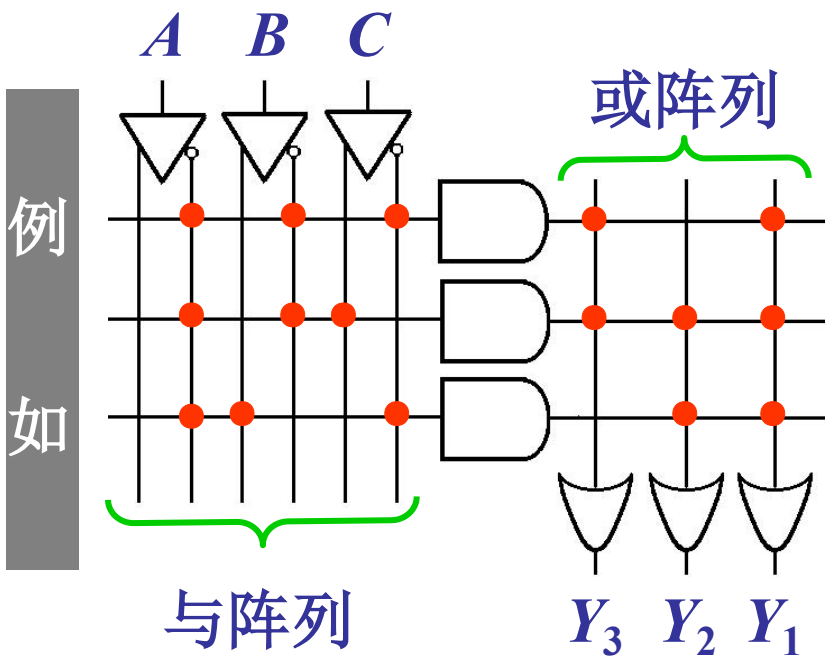
PLD的基本结构

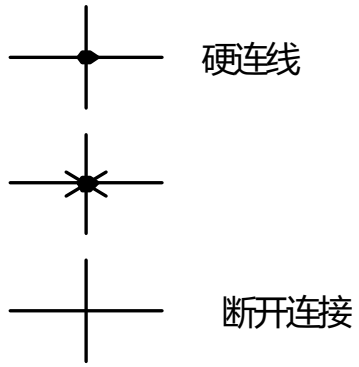


器件关键：可编程的与阵列、或阵列。

1. 基本门电路的PLD表示法

与阵列：N个输入变量，有2N条列线，一个与项最多有N个编程点。



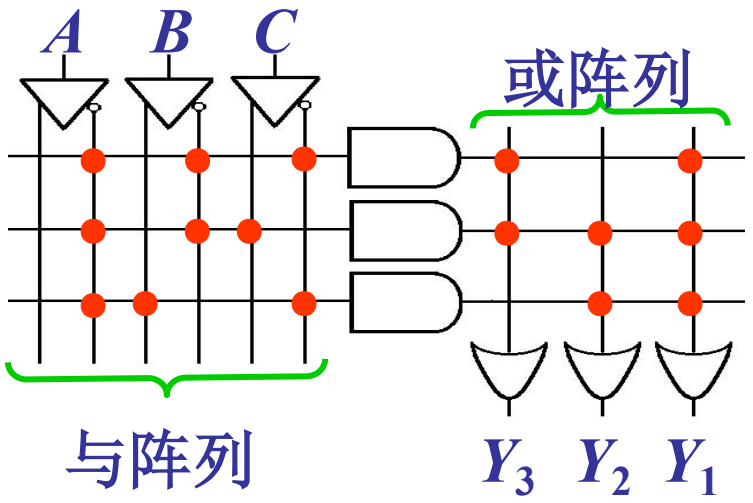


由图可得

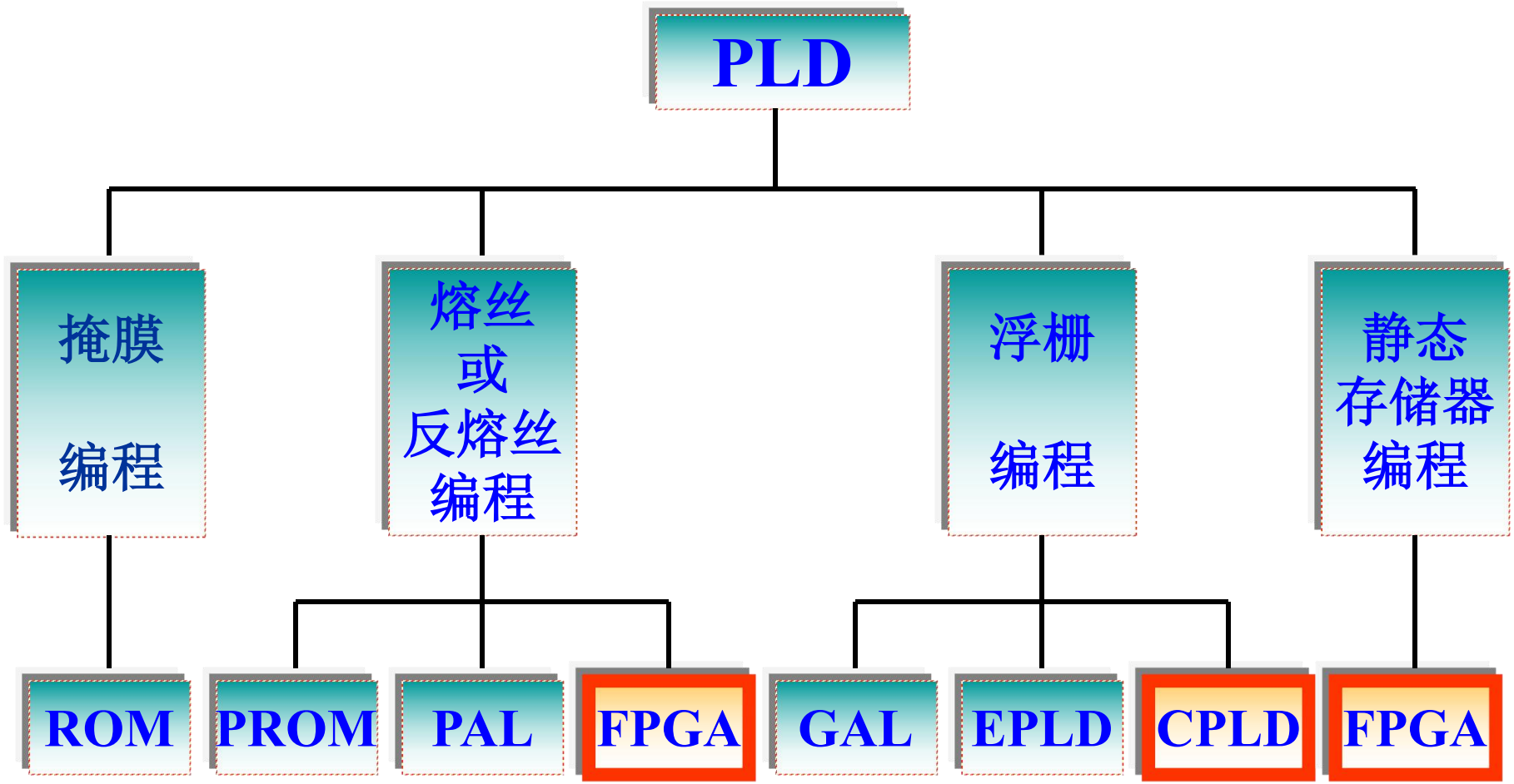
$$\begin{cases} Y_1 = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}B\bar{C} \\ Y_2 = \bar{A}\bar{B}C + \bar{A}B\bar{C} \\ Y_3 = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C \end{cases}$$

2 按可编程的部位分类PLD:

类型	与阵列	或阵列	输出电路
PROM (即可编程 ROM)	固定	可编程	固定
PLA (即 Programmable Logic Array，可编程逻辑阵列)	可编程	可编程	固定
PAL (即 Programmable Array Logic，可编程阵列逻辑)	可编程	固定	固定
GAL (即Generic Array Logic，通用阵列逻辑)	可编程	固定	可组态



按编程方法分类PLD:

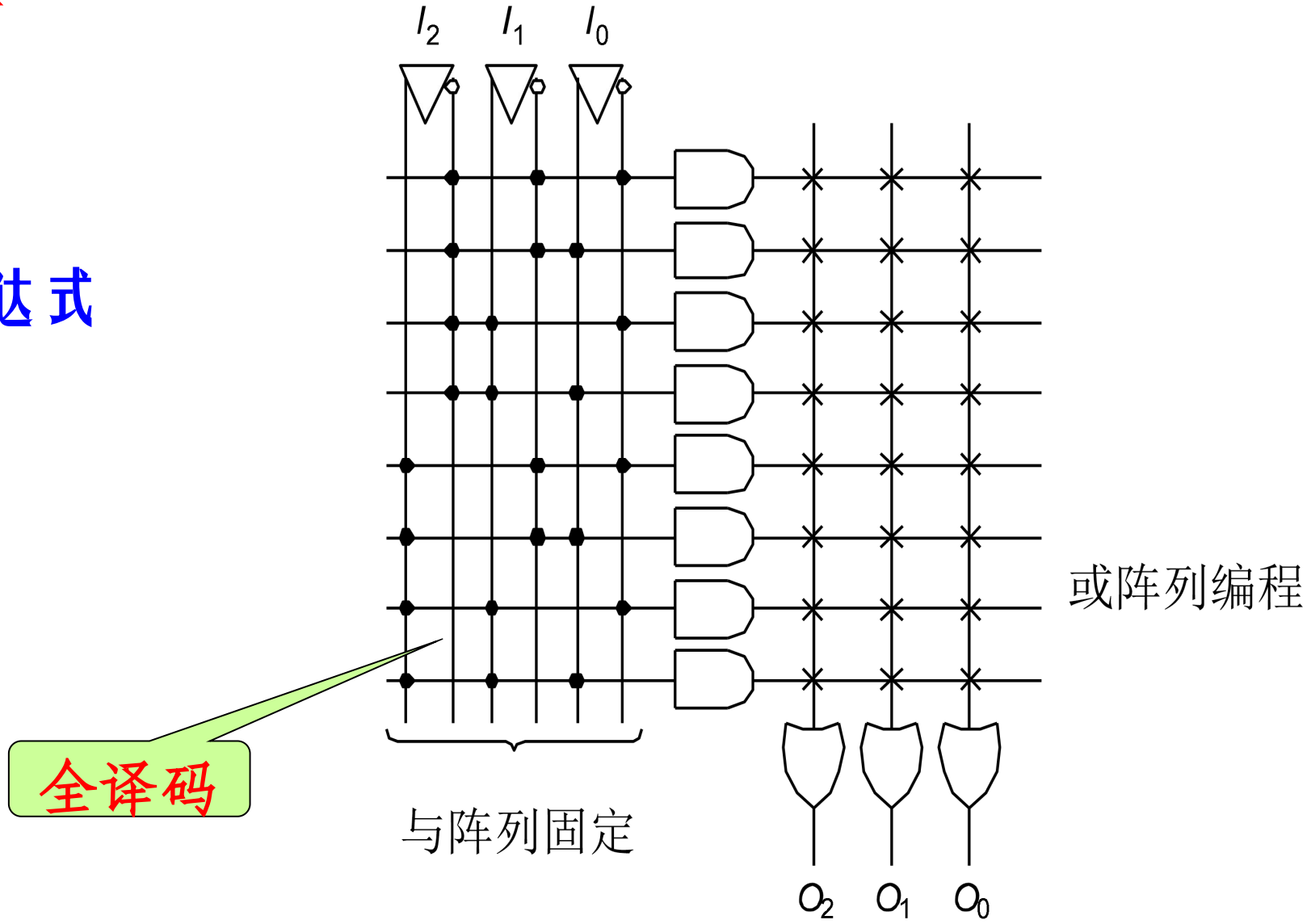


先设计、再仿真，看占用资源数，最后去买器件.

2. 可编程器件的PLD表示

1) PROM

实现组合逻辑的最小项表达式



用PROM构成一个将 4 位二进制码转换为格雷码的逻辑电路。

需要ROM容量：

16×4

列转换的真值表

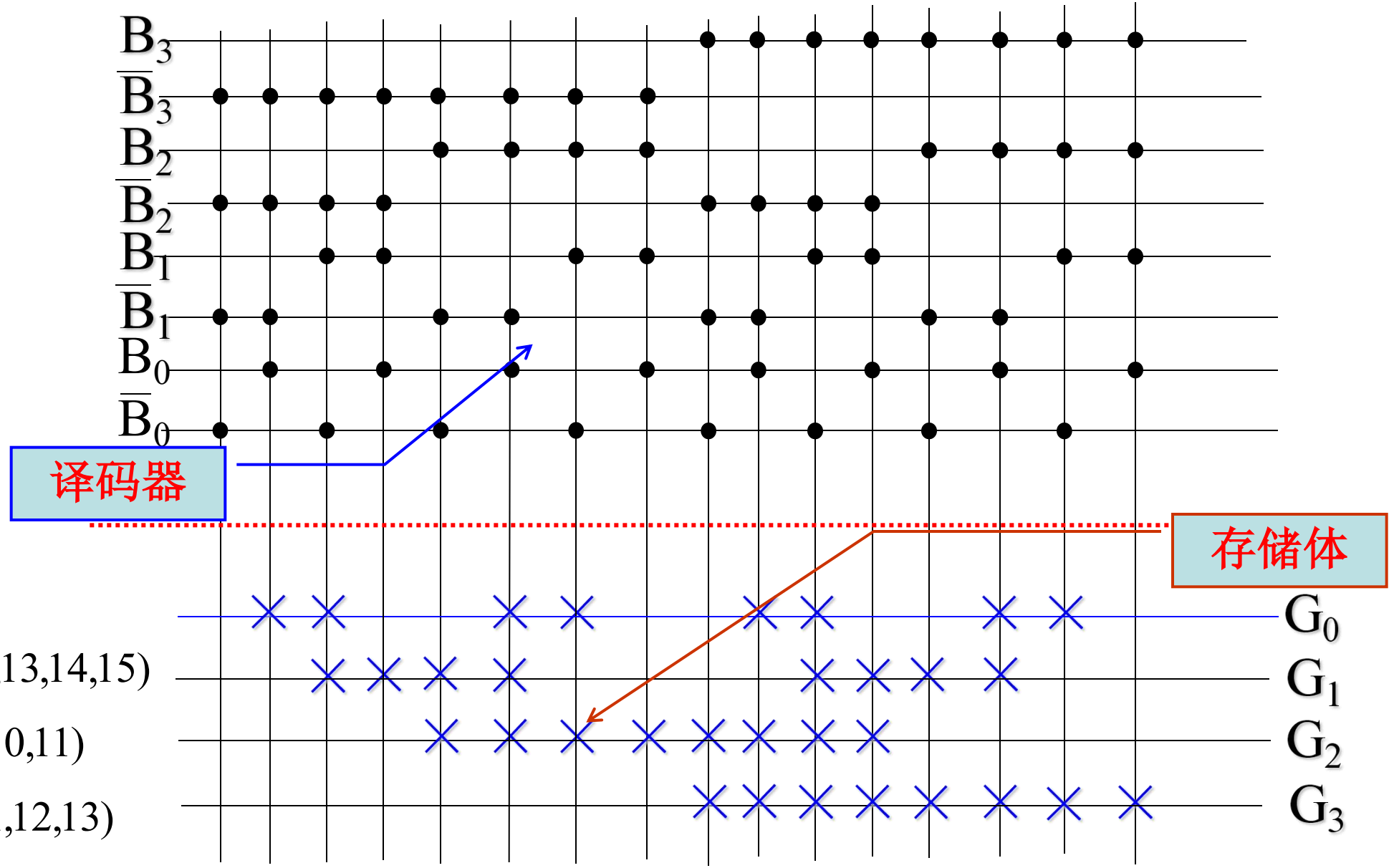
$$G_3 = \sum m(8,9,10,11,12,13,14,15)$$

$$G_2 = \sum m(4,5,6,7,8,9,10,11)$$

$$G_1 = \sum m(2,3,4,5,10,11,12,13)$$

$$G_0 = \sum m(1,2,5,6,9,10,13,14)$$

B3	B2	B1	B0	G_3	G_2	G_1	G_0
0	0	0	0	**0**	**0**	**0**	**0**
0	0	0	1	**0**	**0**	**0**	**1**
0	0	1	0	**0**	**0**	**1**	**1**
0	0	1	1	**0**	**0**	**1**	**0**
0	1	0	0	**0**	**1**	**1**	**0**
0	1	0	1	**0**	**1**	**1**	**1**
0	1	1	0	**0**	**1**	**0**	**1**
0	1	1	1	**0**	**1**	**0**	**0**
1	0	0	0	**1**	**1**	**0**	**0**
1	0	0	1	**1**	**1**	**0**	**1**
1	0	1	0	**1**	**1**	**1**	**1**
1	0	1	1	**1**	**1**	**1**	**0**
1	1	0	0	**1**	**0**	**1**	**0**
1	1	0	1	**1**	**0**	**1**	**1**
1	1	1	0	**1**	**0**	**0**	**1**
1	1	1	1	**1**	**0**	**0**	**0**



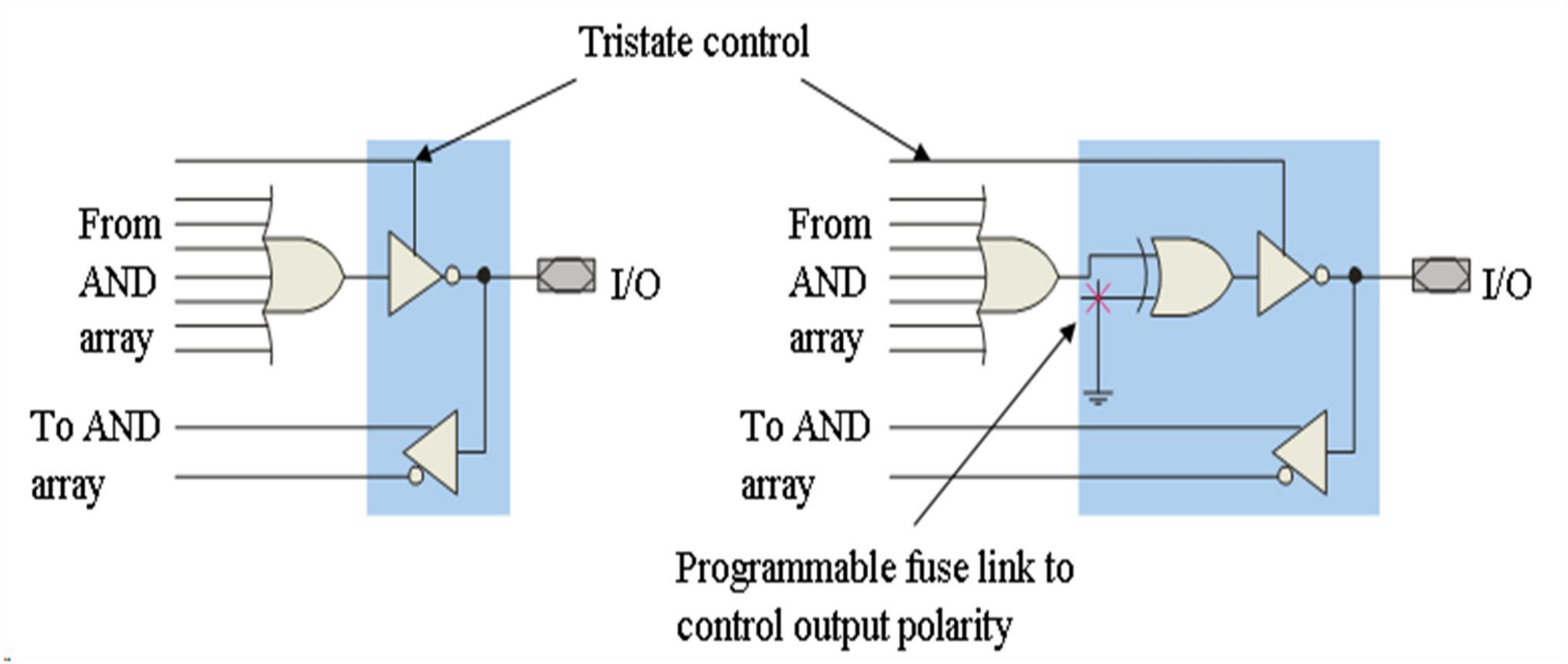
$$G_3 = \sum m(8,9,10,11,12,13,14,15)$$

$$G_2 = \sum m(4,5,6,7,8,9,10,11)$$

$$G_1 = \sum m(2,3,4,5,10,11,12,13)$$

$$G_0 = \sum m(1,2,5,6,9,10,13,14)$$

PALS and GALs



macrocell

§5.2 CPLD、FPGA

目前，使用较广泛的PLD有CPLD和FPGA两大类。

CPLD器件

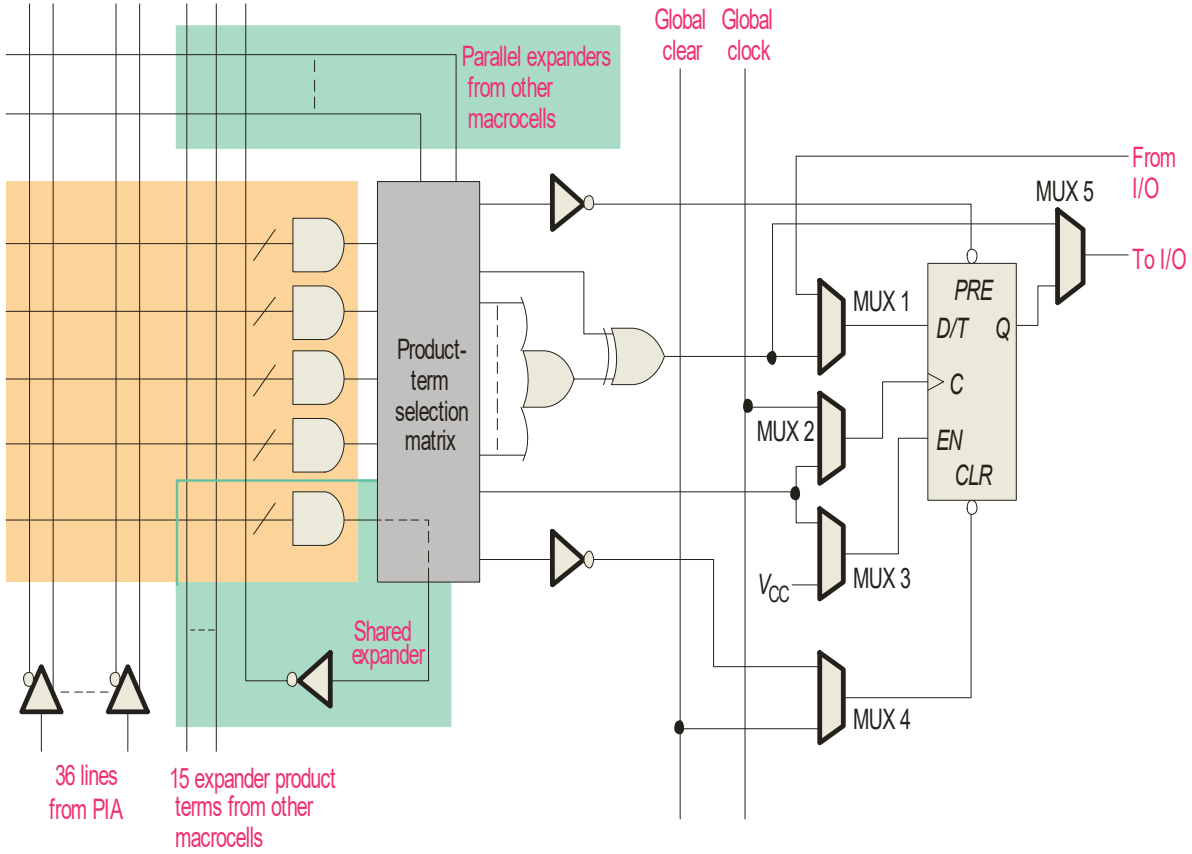


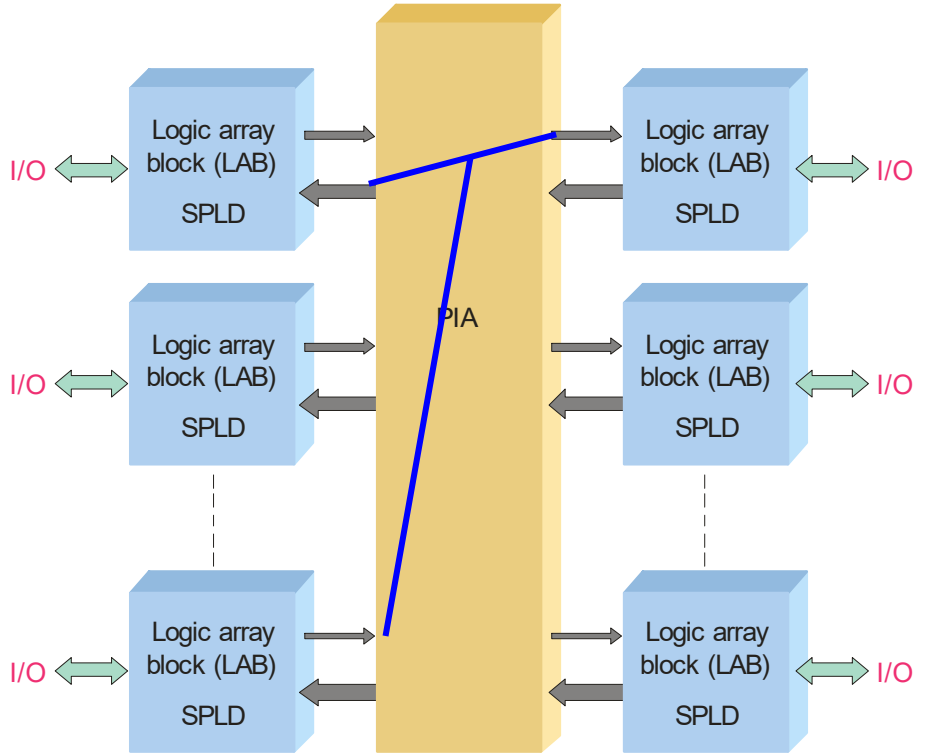
FPGA器件



CPLD

- 可编程逻辑阵列块LAB，Logic Array Block（与或结构较复杂）
- 可编程I/O模块（I/O Cell）
- 可编程内部连线（PIA: programmable interconnect array）（固定长度的金属线）
- 内部延时时间固定，可预测

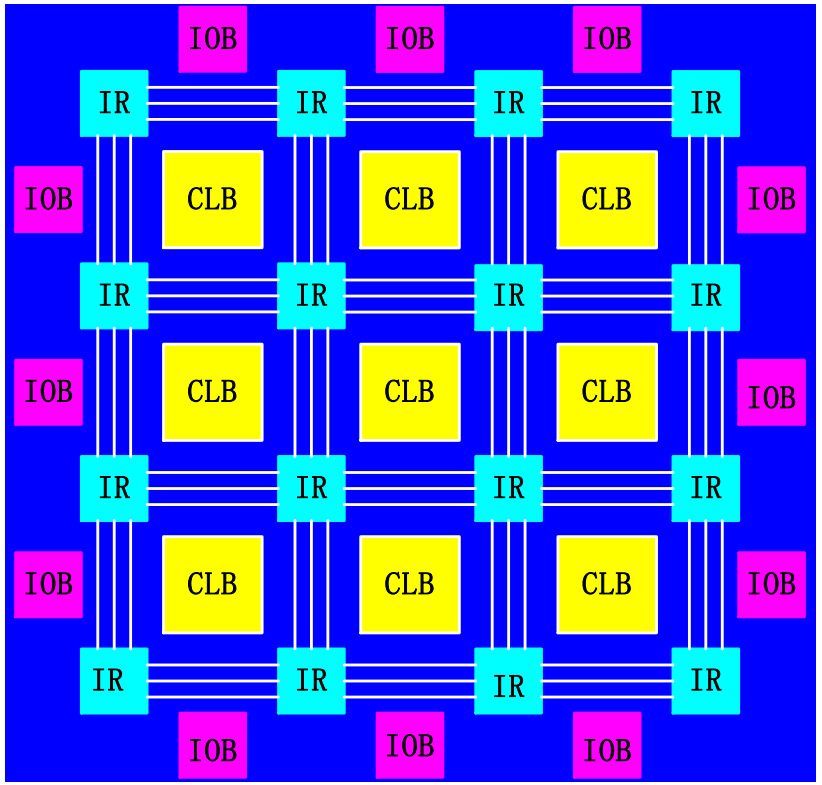




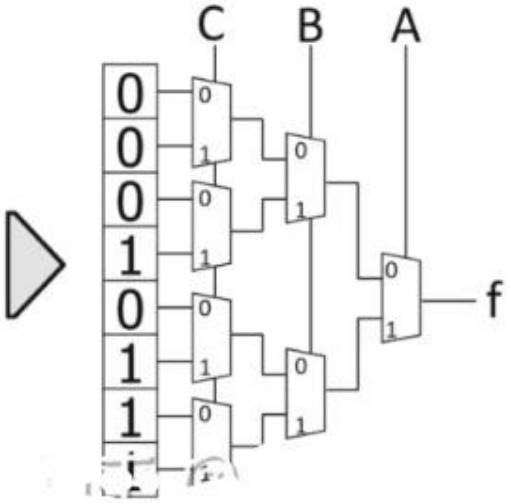
Macrocells 宏单元

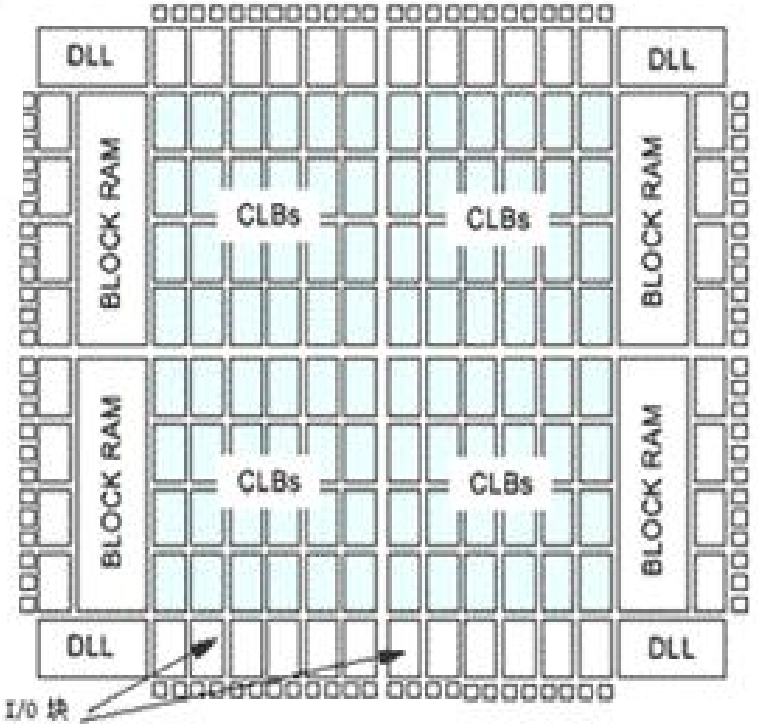
FPGA

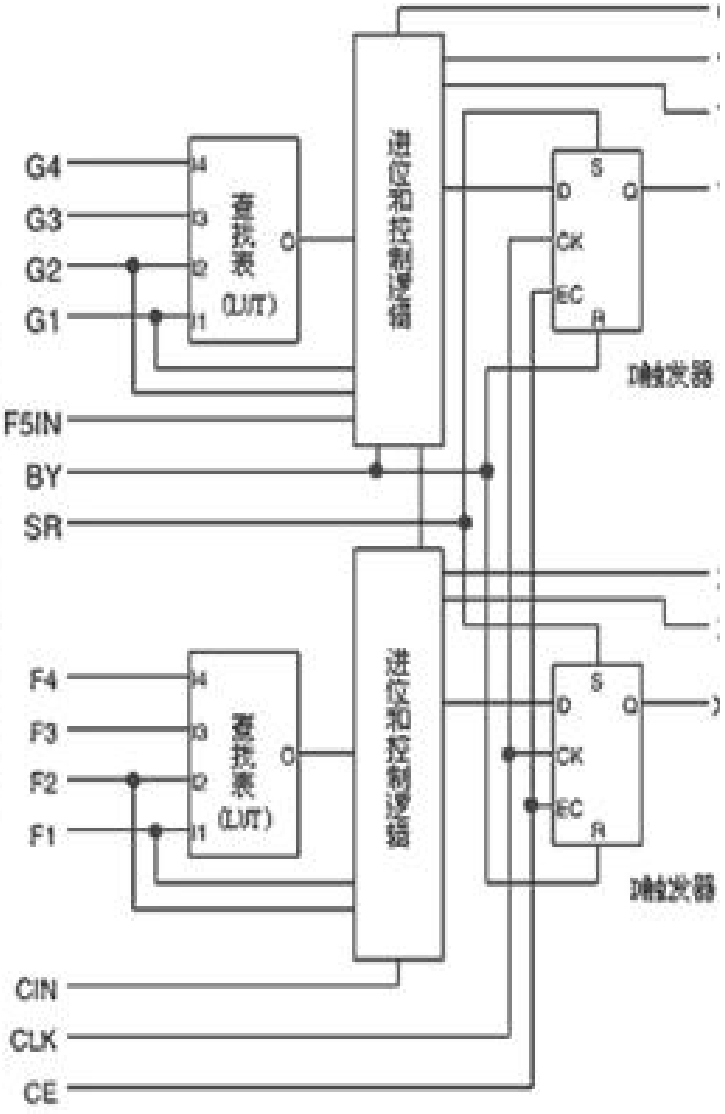
- 可编程逻辑功能块 CLB（Cell Logic Block），实现用户功能的基本单元，由“查找表LUT(look up table)（16×1的SRAM作为函数发生器)+触发器+进位控制逻辑”构成。
- 可编程I/O模块（IOB）
- 可编程互连资源（PIR(global interconnects、 local interconnects））（不同长度的金属线）
- 内部延时时间不固定，预测性差



A	B	C	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1







	CPLD	FPGA
内部结构	Product－term	Look－up Table
程序存储	内部E^2PROM(flash)	SRAM，外挂E^2PROM
资源类型	组合电路资源丰富	触发器资源丰富
集成度	低	高
使用场合	完成控制逻辑	能完成比较复杂的算法
速度	慢	快
其他资源	－	EAB（Embeded Array Block 嵌入式阵列块），锁相环
保密性	可加密	一般不能保密

§5.3 在系统可编程ISP

一. 在系统可编程技术（ISP）

- 传统的PLD在用于生产时，是先编程后装配。
- isp则可以在装配之前、装配过程中和装配之后再编程。

